

Credit Card Users Churn Prediction

Problem Statement

Indented block

Business Context

The Thera bank recently saw a steep decline in the number of users of their credit card, credit cards are a good source of income for banks because of different kinds of fees charged by the banks like annual fees, balance transfer fees, and cash advance fees, late payment fees, foreign transaction fees, and others. Some fees are charged to every user irrespective of usage, while others are charged under specified circumstances.

Customers' leaving credit cards services would lead bank to loss, so the bank wants to analyze the data of customers and identify the customers who will leave their credit card services and reason for same – so that bank could improve upon those areas

You as a Data scientist at Thera bank need to come up with a classification model that will help the bank improve its services so that customers do not renounce their credit cards

Data Description

- CLIENTNUM: Client number. Unique identifier for the customer holding the account
- Attrition_Flag: Internal event (customer activity) variable - if the account is closed then "Attrited Customer" else "Existing Customer"
- Customer_Age: Age in Years
- Gender: Gender of the account holder
- Dependent_count: Number of dependents
- Education_Level: Educational Qualification of the account holder - Graduate, High School, Unknown, Uneducated, College(refers to college student), Post-Graduate, Doctorate
- Marital_Status: Marital Status of the account holder
- Income_Category: Annual Income Category of the account holder
- Card_Category: Type of Card
- Months_on_book: Period of relationship with the bank (in months)
- Total_Relationship_Count: Total no. of products held by the customer
- Months_Inactive_12_mon: No. of months inactive in the last 12 months
- Contacts_Count_12_mon: No. of Contacts in the last 12 months
- Credit_Limit: Credit Limit on the Credit Card
- Total_Revolving_Bal: Total Revolving Balance on the Credit Card
- Avg_Open_To_Buy: Open to Buy Credit Line (Average of last 12 months)
- Total_Amt_Chng_Q4_Q1: Change in Transaction Amount (Q4 over Q1)
- Total_Trans_Amt: Total Transaction Amount (Last 12 months)
- Total_Trans_Ct: Total Transaction Count (Last 12 months)
- Total_Ct_Chng_Q4_Q1: Change in Transaction Count (Q4 over Q1)
- Avg_Utilization_Ratio: Average Card Utilization Ratio

What Is a Revolving Balance?

- If we don't pay the balance of the revolving credit account in full every month, the unpaid portion carries over to the next month. That's called a revolving balance

What is the Average Open to buy?

- 'Open to Buy' means the amount left on your credit card to use. Now, this column represents the average of this value for the last 12 months.

What is the Average utilization Ratio?

- The Avg_Utilization_Ratio represents how much of the available credit the customer spent. This is useful for calculating credit scores.

Relation b/w Avg_Open_To_Buy, Credit_Limit and Avg_Utilization_Ratio:

- $(\text{Avg_Open_To_Buy} / \text{Credit_Limit}) + \text{Avg_Utilization_Ratio} = 1$

Importing necessary libraries

- List item
- List item

```
In [1]: import pandas as pd
import numpy as np
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (
    AdaBoostClassifier,
    GradientBoostingClassifier,
    BaggingClassifier,
)
from xgboost import XGBClassifier
from sklearn import metrics
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.impute import KNNImputer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso
from sklearn.metrics import r2_score
from sklearn import preprocessing
from sklearn.preprocessing import PolynomialFeatures
from pandas import read_csv
from sklearn.model_selection import train_test_split
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score

from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from imblearn.over_sampling import RandomOverSampler
from imblearn.under_sampling import TomekLinks
from imblearn.combine import SMOTETomek
from imblearn.under_sampling import ClusterCentroids
from sklearn.preprocessing import StandardScaler, MinMaxScaler, OneHotEncoder
from sklearn.svm import SVC

from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
pd.set_option("display.max_columns", None)
import warnings

warnings.filterwarnings("ignore")
```

Loading Data

```
In [3]: from google.colab import drive
drive.mount('/content/drive')
# read the data
df = pd.read_csv('/content/drive/MyDrive/BankChurners.csv')
```

Mounted at /content/drive

```
In [4]: df.shape
```

```
Out[4]: (10127, 21)
```

- The dataset has 2240 rows and 22 columns

Data Overview

```
In [5]: data = df.copy()
```

```
In [7]: # First 10 rows of the data
data.head(10)
```

Out[7]:	CLIENTNUM	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category
0	768805383	Existing Customer	45	M	3	High School	Married	60K – 80K	Blue
1	818770008	Existing Customer	49	F	5	Graduate	Single	Less than \$40K	Blue
2	713982108	Existing Customer	51	M	3	Graduate	Married	80K – 120K	Blue
3	769911858	Existing Customer	40	F	4	High School	NaN	Less than \$40K	Blue
4	709106358	Existing Customer	40	M	3	Uneducated	Married	60K – 80K	Blue
5	713061558	Existing Customer	44	M	2	Graduate	Married	40K – 60K	Blue
6	810347208	Existing Customer	51	M	4	NaN	Married	\$120K +	Gold
7	818906208	Existing Customer	32	M	0	High School	NaN	60K – 80K	Silver
8	710930508	Existing Customer	37	M	3	Uneducated	Single	60K – 80K	Blue
9	719661558	Existing Customer	48	M	2	Graduate	Single	80K – 120K	Blue

```
In [8]: # Check the data types in the dataset
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10127 entries, 0 to 10126
Data columns (total 21 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   CLIENTNUM                            10127 non-null  int64
1   Attrition_Flag                       10127 non-null  object
2   Customer_Age                         10127 non-null  int64
3   Gender                               10127 non-null  object
4   Dependent_count                      10127 non-null  int64
5   Education_Level                      8608 non-null   object
6   Marital_Status                      9378 non-null   object
7   Income_Category                     10127 non-null  object
8   Card_Category                       10127 non-null  object
9   Months_on_book                      10127 non-null  int64
10  Total_Relationship_Count             10127 non-null  int64
11  Months_Inactive_12_mon              10127 non-null  int64
12  Contacts_Count_12_mon               10127 non-null  int64
13  Credit_Limit                        10127 non-null  float64
14  Total_Revolving_Bal                 10127 non-null  int64
15  Avg_Open_To_Buy                     10127 non-null  float64
16  Total_Amt_Chng_Q4_Q1                10127 non-null  float64
17  Total_Trans_Amt                     10127 non-null  int64
18  Total_Trans_Ct                      10127 non-null  int64
19  Total_Ct_Chng_Q4_Q1                 10127 non-null  float64
20  Avg_Utilization_Ratio                10127 non-null  float64
dtypes: float64(5), int64(10), object(6)
memory usage: 1.6+ MB
```

- There are a total of 21 columns and 10,000+ observations in the dataset
- We can see that 2 columns have around 1,000 non-null values i.e. columns have missing values.

```
In [9]: # duplicate values in the data
data.duplicated().sum()
```

Out[9]: 0

```
In [23]: # Missing values in the data
(data.isnull().sum() / data.isnull().count() * 100, 2)
```

```
Out[23]: (Attrition_Flag          0.000
Customer_Age                0.000
Gender                      0.000
Dependent_count             0.000
Education_Level             15.000
Marital_Status              7.396
Income_Category             10.981
Card_Category               0.000
Months_on_book              0.000
Total_Relationship_Count    0.000
Months_Inactive_12_mon      0.000
Contacts_Count_12_mon       0.000
Credit_Limit                0.000
Total_Revolving_Bal         0.000
Avg_Open_To_Buy             0.000
Total_Amt_Chng_Q4_Q1        0.000
Total_Trans_Amt             0.000
Total_Trans_Ct              0.000
Total_Ct_Chng_Q4_Q1         0.000
Avg_Utilization_Ratio       0.000
dtype: float64,
2)
```

- The Education_Level column has 15% missing values
- The Marital_Status column has 7.40% missing values from the total observations.
- The Income_Category column has 10.98% missing values from the total observations.

```
In [11]: # uniques data
data.nunique()
```

```
Out[11]: CLIENTNUM          10127
Attrition_Flag           2
Customer_Age             45
Gender                   2
Dependent_count          6
Education_Level          6
Marital_Status           3
Income_Category          6
Card_Category            4
Months_on_book           44
Total_Relationship_Count  6
Months_Inactive_12_mon   7
Contacts_Count_12_mon    7
Credit_Limit            6205
Total_Revolving_Bal      1974
Avg_Open_To_Buy          6813
Total_Amt_Chng_Q4_Q1     1158
Total_Trans_Amt          5033
Total_Trans_Ct           126
Total_Ct_Chng_Q4_Q1      830
Avg_Utilization_Ratio    964
dtype: int64
```

```
In [12]: data.describe().T
```

Out[12]:		count	mean	std	min	25%	50%	75%	max
	CLIENTNUM	10127.000	739177606.334	36903783.450	708082083.000	713036770.500	717926358.000	773143533.000	828343083.000
	Customer_Age	10127.000	46.326	8.017	26.000	41.000	46.000	52.000	73.000
	Dependent_count	10127.000	2.346	1.299	0.000	1.000	2.000	3.000	5.000
	Months_on_book	10127.000	35.928	7.986	13.000	31.000	36.000	40.000	56.000
	Total_Relationship_Count	10127.000	3.813	1.554	1.000	3.000	4.000	5.000	6.000
	Months_Inactive_12_mon	10127.000	2.341	1.011	0.000	2.000	2.000	3.000	6.000
	Contacts_Count_12_mon	10127.000	2.455	1.106	0.000	2.000	2.000	3.000	6.000
	Credit_Limit	10127.000	8631.954	9088.777	1438.300	2555.000	4549.000	11067.500	34516.000
	Total_Revolving_Bal	10127.000	1162.814	814.987	0.000	359.000	1276.000	1784.000	2517.000
	Avg_Open_To_Buy	10127.000	7469.140	9090.685	3.000	1324.500	3474.000	9859.000	34516.000
	Total_Amt_Chng_Q4_Q1	10127.000	0.760	0.219	0.000	0.631	0.736	0.859	3.359
	Total_Trans_Amt	10127.000	4404.086	3397.129	510.000	2155.500	3899.000	4741.000	18484.000
	Total_Trans_Ct	10127.000	64.859	23.473	10.000	45.000	67.000	81.000	139.000
	Total_Ct_Chng_Q4_Q1	10127.000	0.712	0.238	0.000	0.582	0.702	0.818	3.712
	Avg_Utilization_Ratio	10127.000	0.275	0.276	0.000	0.023	0.176	0.503	0.950

```
In [ ]: # non-numerical columns in the data
data.describe(exclude=np.number).T
```

	count	unique	top	freq
Attrition_Flag	10127	2	Existing Customer	8500
Gender	10127	2	F	5358
Education_Level	8608	6	Graduate	3128
Marital_Status	9378	3	Married	4687
Income_Category	10127	6	Less than \$40K	3561
Card_Category	10127	4	Blue	9436

Data Pre-processing

CLIENTNUM is unique for each customer and so we drop it.

```
In [13]: data.drop(columns=["CLIENTNUM"], inplace=True)
```

```
In [14]: # Making a list of all catrgorical variables
cat_col = data.select_dtypes("object").columns.to_list()
for column in cat_col:
    print(data[column].value_counts())
    print("-" * 40)
```

```
Existing Customer    8500
Attrited Customer    1627
Name: Attrition_Flag, dtype: int64
-----
F         5358
M         4769
Name: Gender, dtype: int64
-----
Graduate         3128
High School      2013
Uneducated       1487
College          1013
Post-Graduate    516
Doctorate        451
Name: Education_Level, dtype: int64
-----
Married         4687
Single          3943
Divorced         748
Name: Marital_Status, dtype: int64
-----
Less than $40K    3561
$40K - $60K      1790
$80K - $120K     1535
$60K - $80K      1402
abc              1112
$120K +          727
Name: Income_Category, dtype: int64
-----
Blue           9436
Silver         555
Gold           116
Platinum        20
Name: Card_Category, dtype: int64
-----
```

```
In [15]: # Get value counts
df["Attrition_Flag"].value_counts(normalize=True)
```

```
Out[15]: Existing Customer    0.839
Attrited Customer    0.161
Name: Attrition_Flag, dtype: float64
```

- We have more female customers as compared to male customers.
- There are very few observations of Platinum Card holders (20).
- We can see that the distribution of classes in the target variable is imbalanced i.e. only ~16% observations with existing customers (non-attritees).

Replace incorrect data entry values with an imputed mode.

```
In [ ]: data[data.Income_Category == "abc"]
```

Out[]:	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category	Months_o
	19	Existing Customer	45	F	2	Graduate	Married	abc	Blue
	28	Existing Customer	44	F	3	Uneducated	Single	abc	Blue
	39	Attrited Customer	66	F	0	Doctorate	Married	abc	Blue
	44	Existing Customer	38	F	4	Graduate	Single	abc	Blue
	58	Existing Customer	44	F	5	Graduate	Married	abc	Blue
	...	...	...	...	...	...	...	...	...
	10021	Attrited Customer	30	F	1	Graduate	Married	abc	Blue
	10040	Attrited Customer	50	F	3	Doctorate	Single	abc	Blue
	10083	Existing Customer	42	F	4	Uneducated	Married	abc	Blue
	10092	Attrited Customer	40	F	3	Graduate	Married	abc	Blue
	10119	Attrited Customer	55	F	3	Uneducated	Single	abc	Blue

1112 rows × 20 columns

In [16]:	# replacing values with missing data.Income_Category.replace(to_replace="abc", value=np.nan, inplace=True)
In [17]:	data[data.Income_Category == "abc"]
Out[17]:	Attrition_Flag Customer_Age Gender Dependent_count Education_Level Marital_Status Income_Category Card_Category Months_on_boo

In []:	data.Income_Category = data.Income_Category.fillna(value=data["Income_Category"].value_counts().index[0])
In []:	# checking values data.Income_Category.value_counts()
Out[]:	Less than \$40K 4673 \$40K - \$60K 1790 \$80K - \$120K 1535 \$60K - \$80K 1402 \$120K + 727 Name: Income_Category, dtype: int64

Exploratory Data Analysis (EDA)

- EDA is an important part of any project involving data.
- It is important to investigate and understand the data better before building a model with it.
- A few questions have been mentioned below which will help you approach the analysis in the right manner and generate insights from the data.
- A thorough analysis of the data, in addition to the questions mentioned below, should be done.

Questions:

1. How is the total transaction amount distributed?
2. What is the distribution of the level of education of customers?
3. What is the distribution of the level of income of customers?
4. How does the change in transaction amount between Q4 and Q1 (total_ct_change_Q4_Q1) vary by the customer's account status (Attrition_Flag)?
5. How does the number of months a customer was inactive in the last 12 months (Months_Inactive_12_mon) vary by the customer's account status (Attrition_Flag)?
6. What are the attributes that have a strong correlation with each other?

The below functions need to be defined to carry out the

1. List item
2. List item

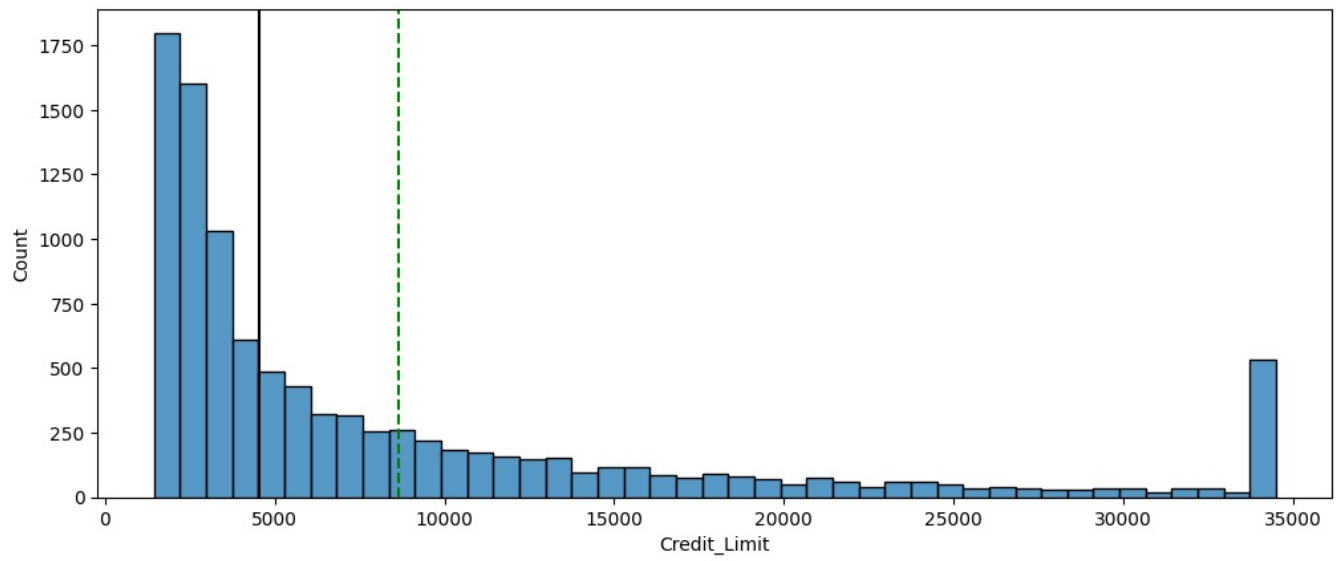
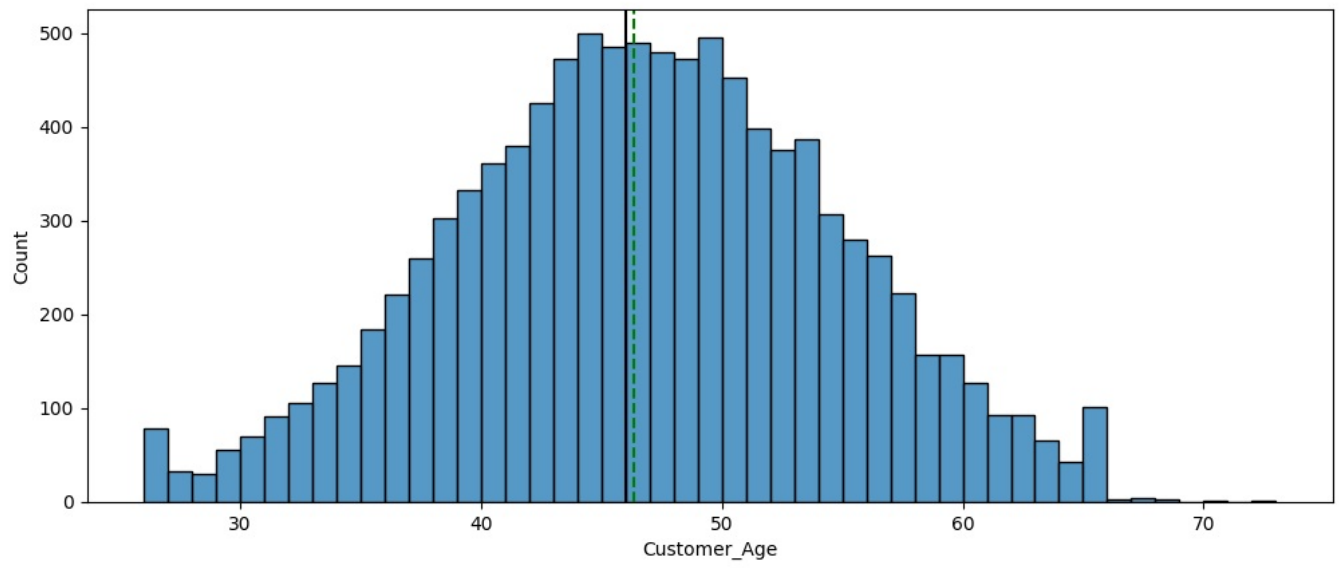
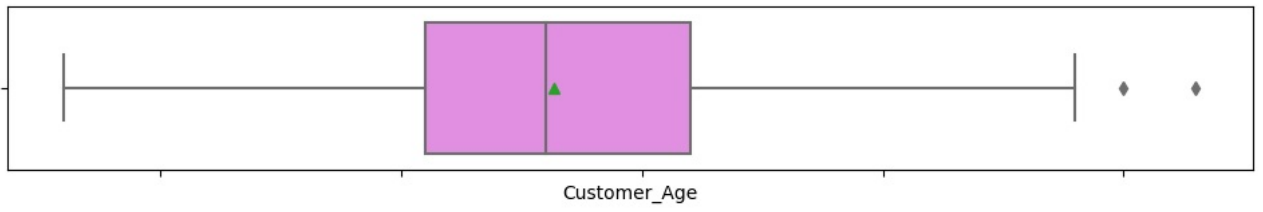
Exploratory Data Analysis.

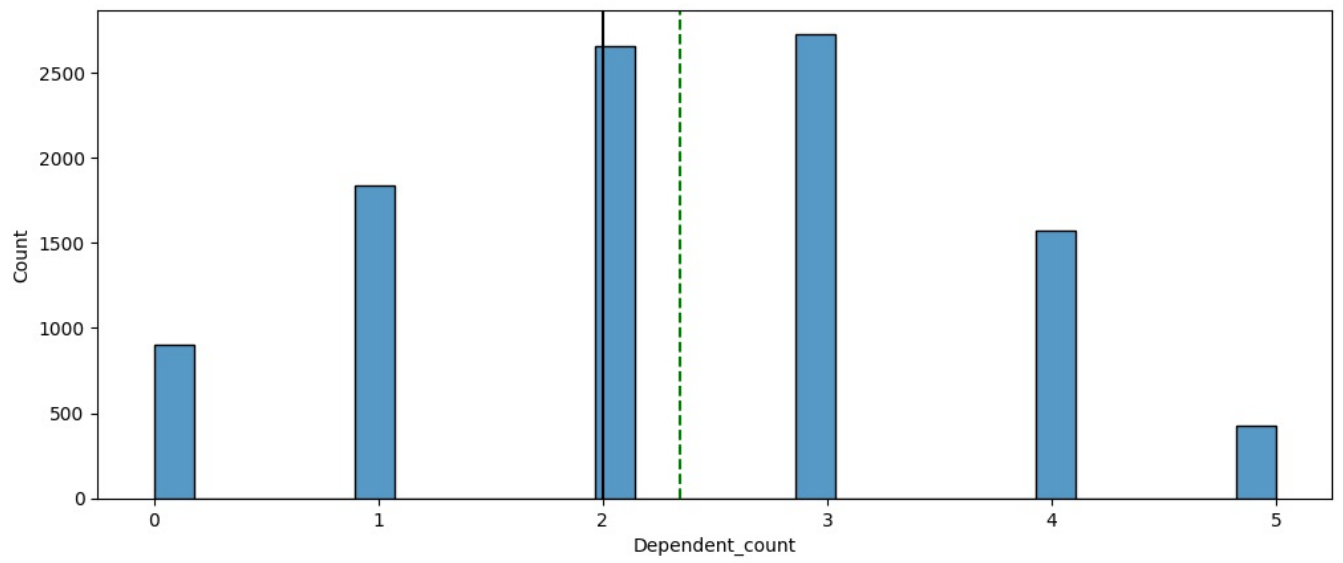
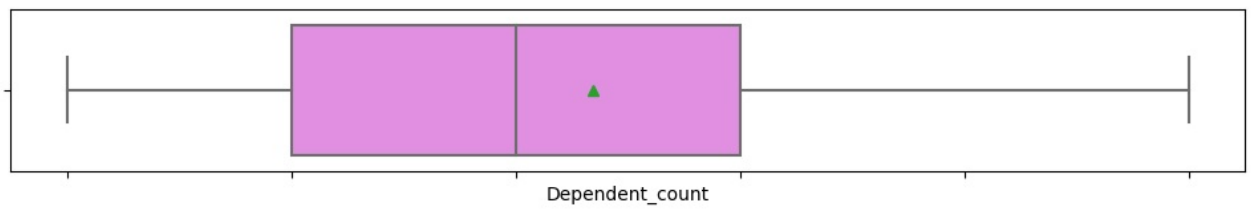
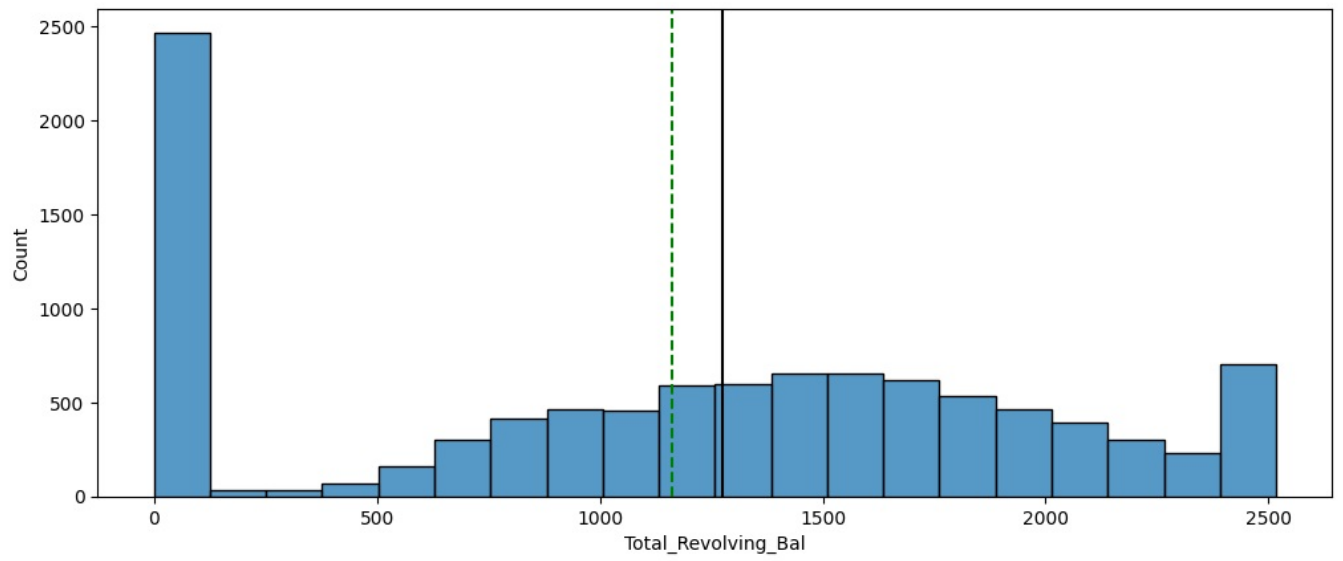
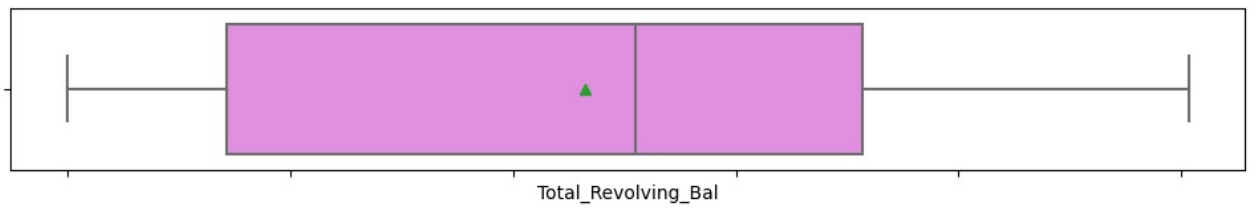
```
In [18]: # function to plot a boxplot and a histogram along the same scale.
```

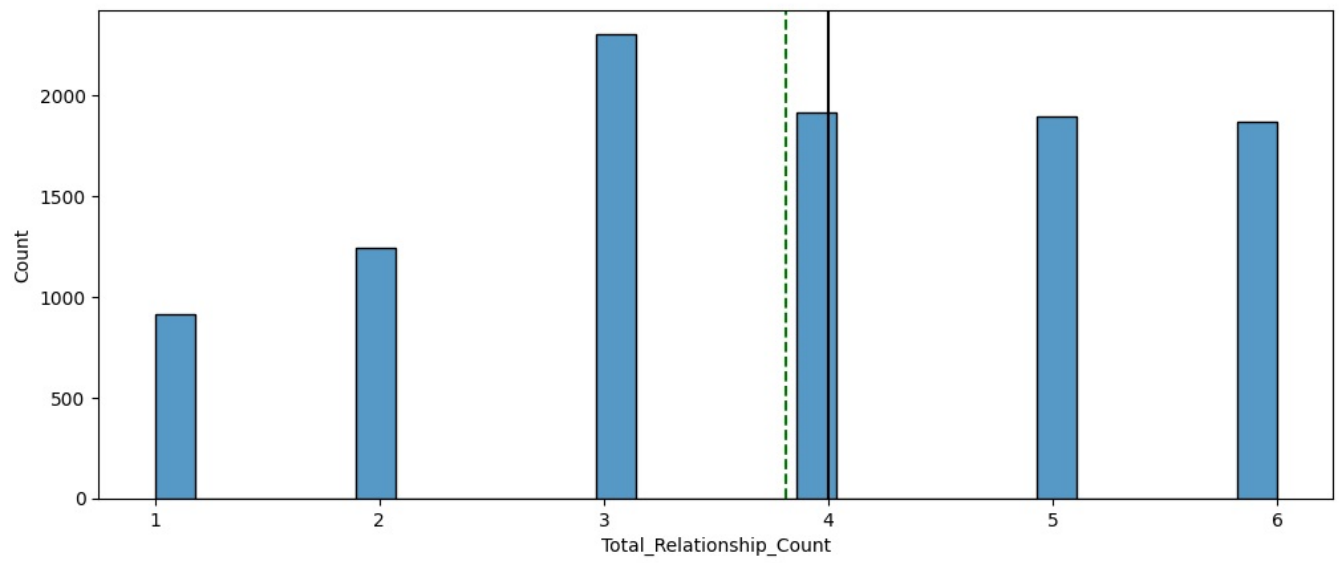
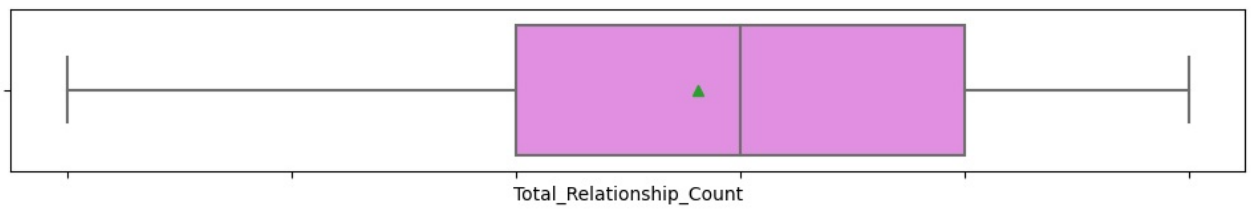
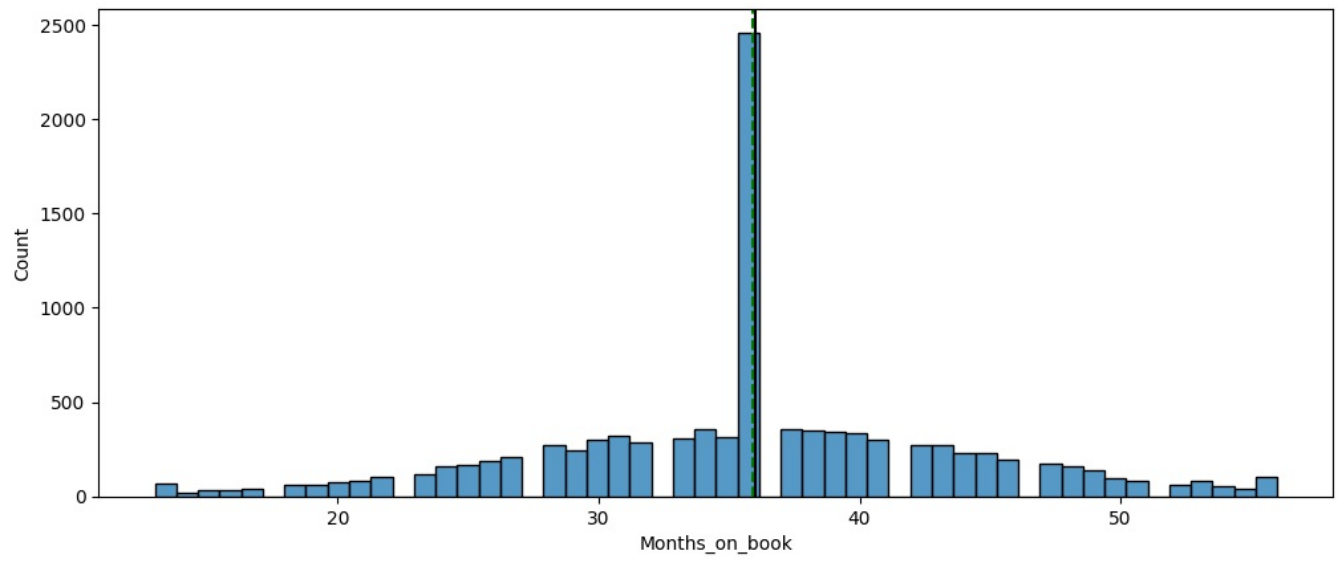
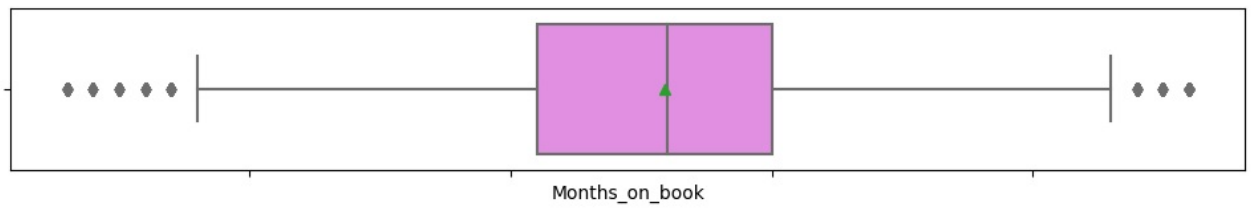
```
def histogram_boxplot(data, feature, figsize=(12, 7), kde=False, bins=None):
    """
    Boxplot and histogram combined

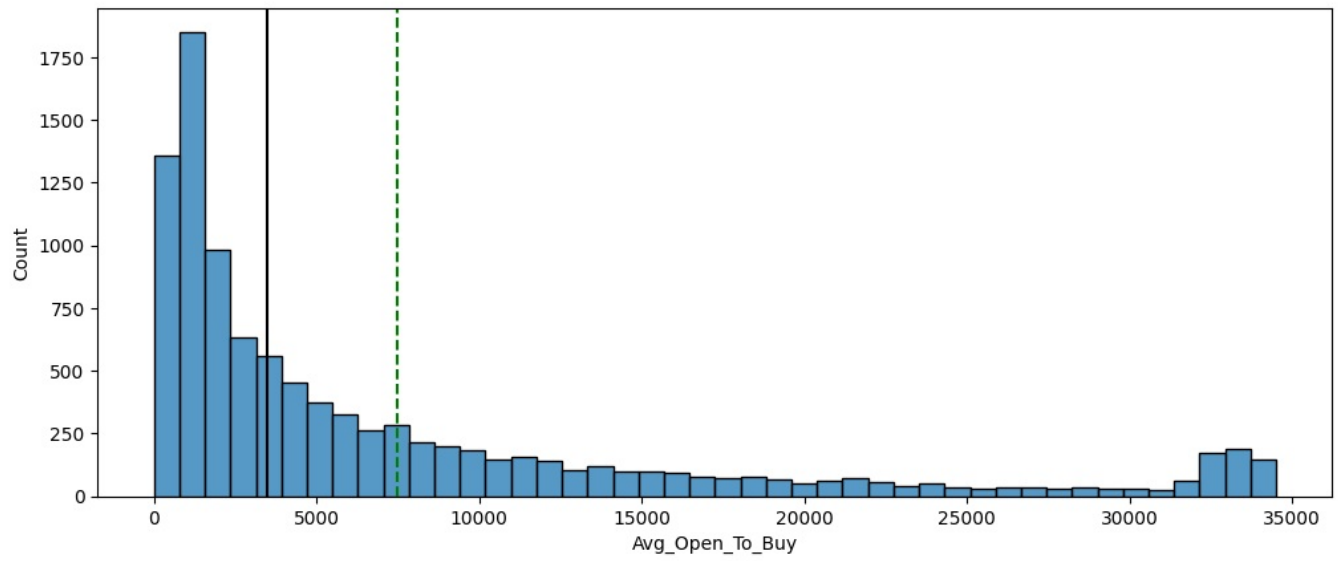
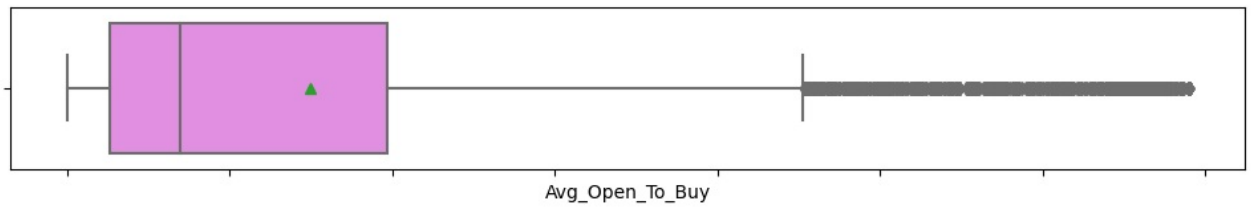
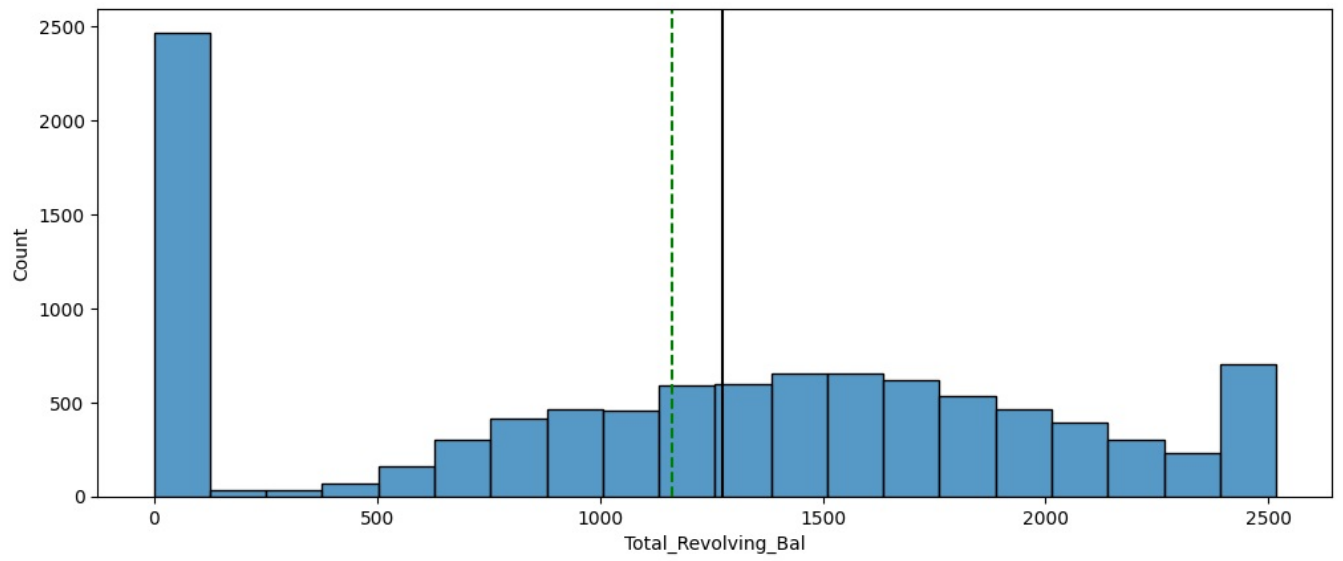
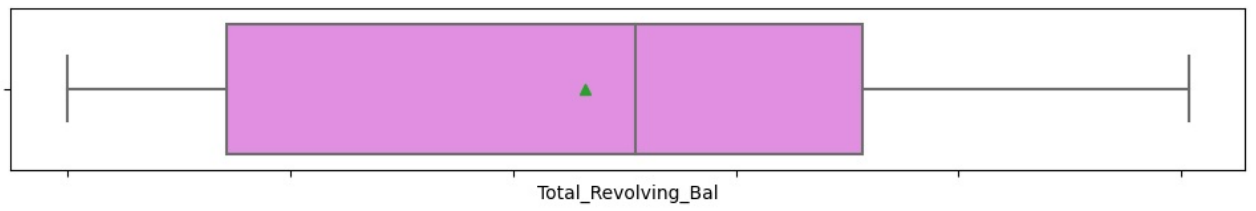
    data: dataframe
    feature: dataframe column
    figsize: size of figure (default (12,7))
    kde: whether to the show density curve (default False)
    bins: number of bins for histogram (default None)
    """
    f2, (ax_box2, ax_hist2) = plt.subplots(
        nrows=2, # Number of rows of the subplot grid= 2
        sharex=True, # x-axis will be shared among all subplots
        gridspec_kw={"height_ratios": (0.25, 0.75)},
        figsize=figsize,
    ) # creating the 2 subplots
    sns.boxplot(
        data=data, x=feature, ax=ax_box2, showmeans=True, color="violet"
    ) # boxplot will be created and a triangle will indicate the mean value of the column
    sns.histplot(
        data=data, x=feature, kde=kde, ax=ax_hist2, bins=bins, palette="winter"
    ) if bins else sns.histplot(
        data=data, x=feature, kde=kde, ax=ax_hist2
    ) # For histogram
    ax_hist2.axvline(
        data[feature].mean(), color="green", linestyle="--"
    ) # Add mean to the histogram
    ax_hist2.axvline(
        data[feature].median(), color="black", linestyle="-."
    ) # Add median to the histogram
```

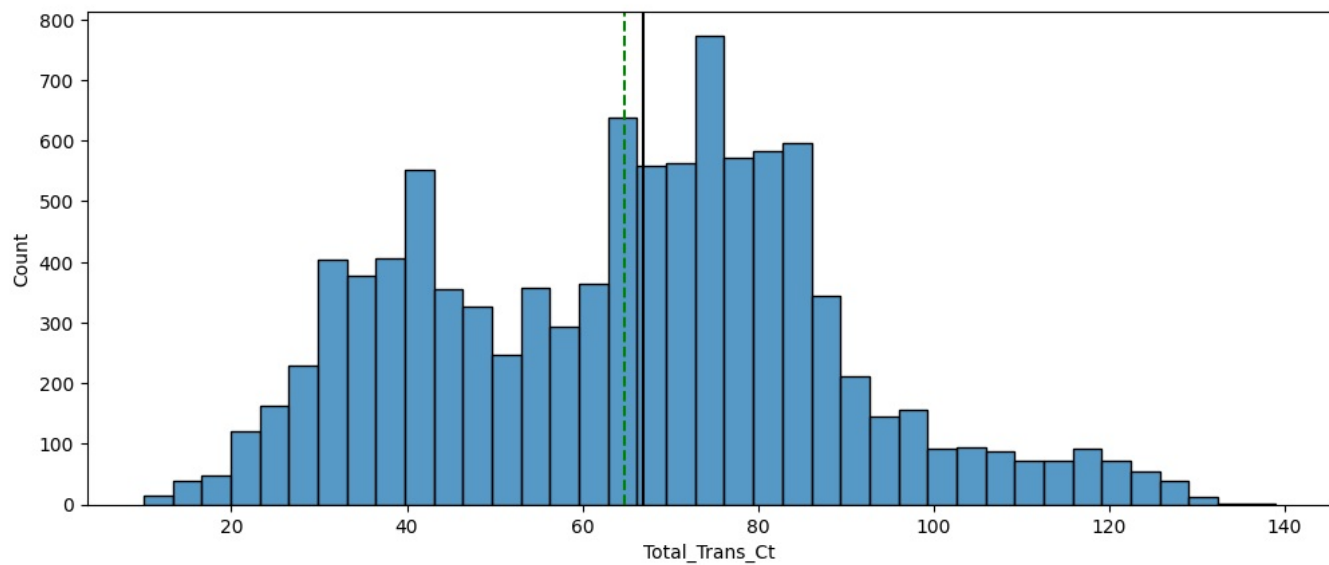
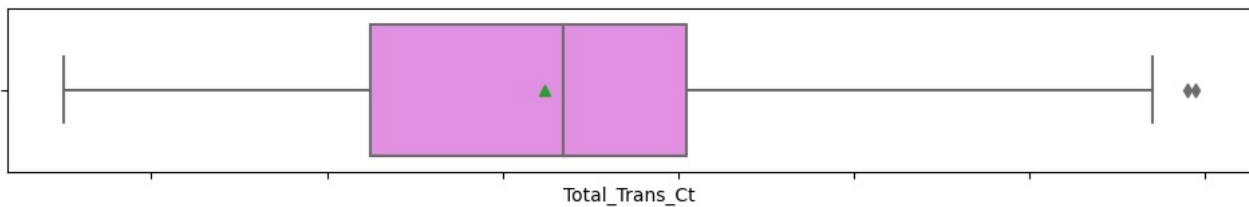
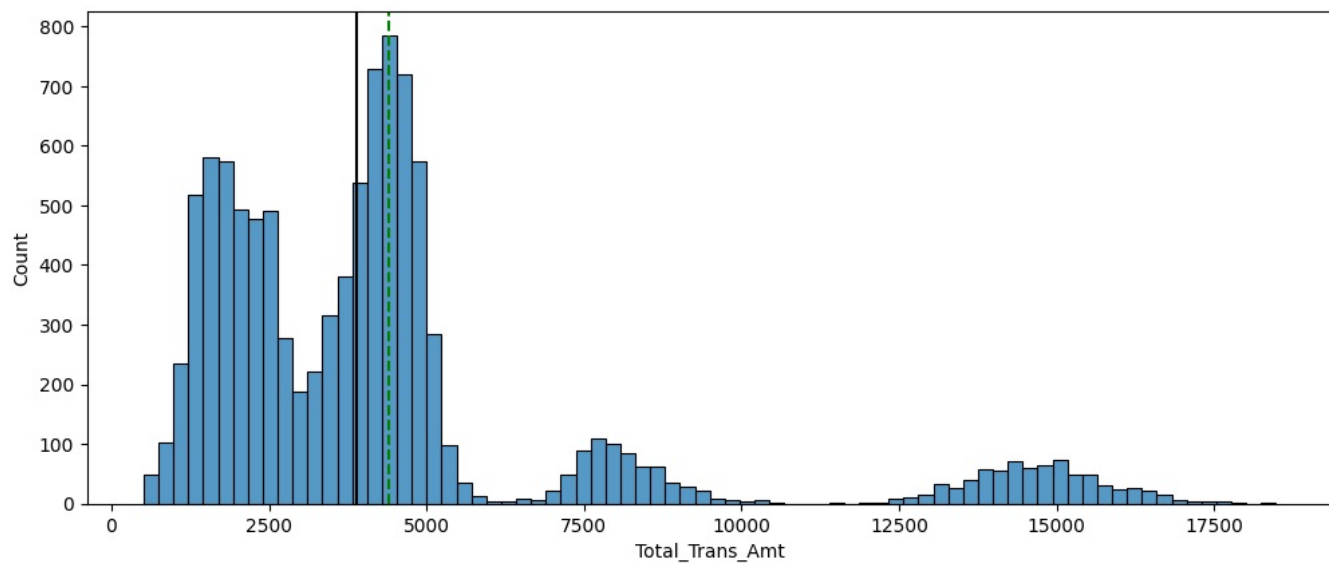
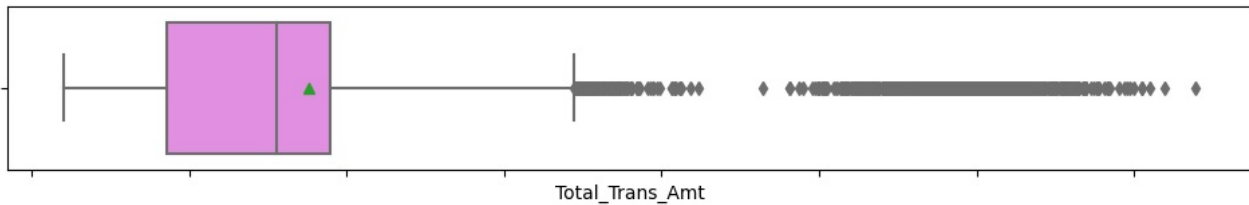
```
In [19]: # Observations on Customer_age
histogram_boxplot(data, "Customer_Age")
# observations on Income
histogram_boxplot(data, "Credit_Limit")
# observations on balance
histogram_boxplot(data, "Total_Revolving_Bal")
# observations on dependents
histogram_boxplot(data, "Dependent_count")
# observations on book months
histogram_boxplot(data, "Months_on_book")
# observations on relationship count
histogram_boxplot(data, "Total_Relationship_Count")
# Checking 10 largest values of amount of credit limit
data.Total_Revolving_Bal.nlargest(10)
# observations on MntFishProducts
histogram_boxplot(data, "Total_Revolving_Bal")
# observations on Avg_Open_To_Buy
histogram_boxplot(data, "Avg_Open_To_Buy")
# observations on Total_Trans_Amt
histogram_boxplot(data, "Total_Trans_Amt")
# observations on Total_Trans_Ct
histogram_boxplot(data, "Total_Trans_Ct")
# observations on Total_Ct_Chng_Q4_Q1
histogram_boxplot(data, "Total_Ct_Chng_Q4_Q1")
# observations on Total_Amt_Chng_Q4_Q1
histogram_boxplot(data, "Total_Amt_Chng_Q4_Q1")
# Observations on Avg_Utilization_Ratio
histogram_boxplot(data, "Avg_Utilization_Ratio")
```

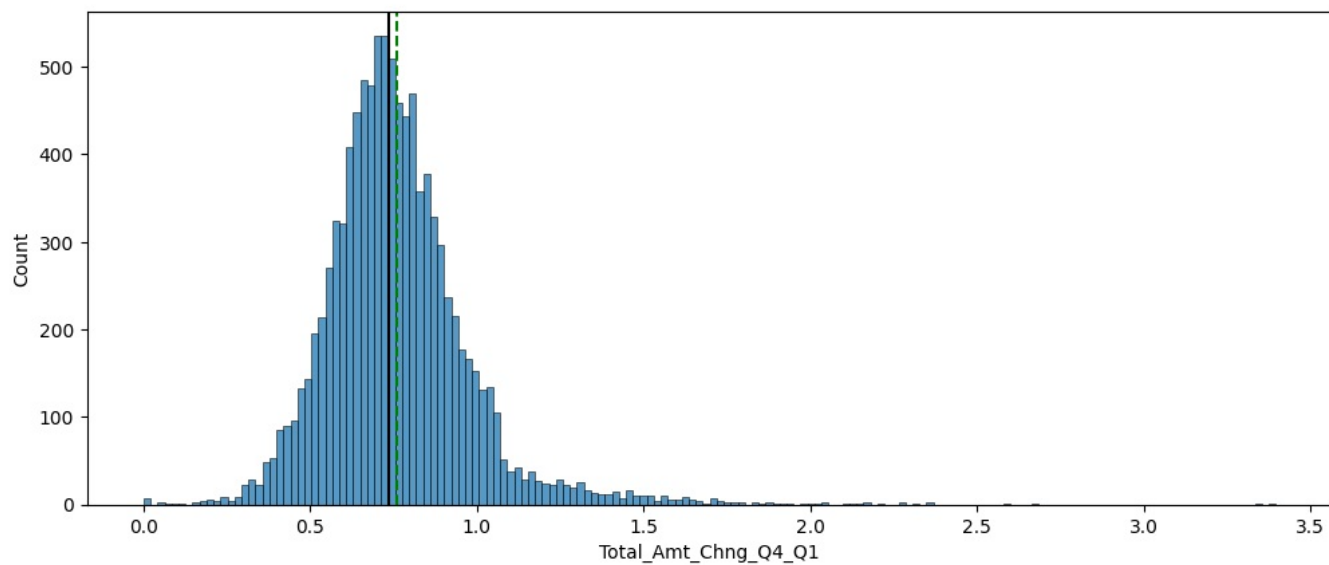
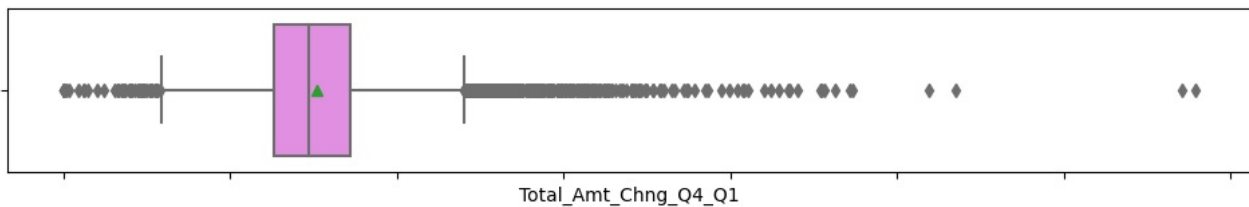
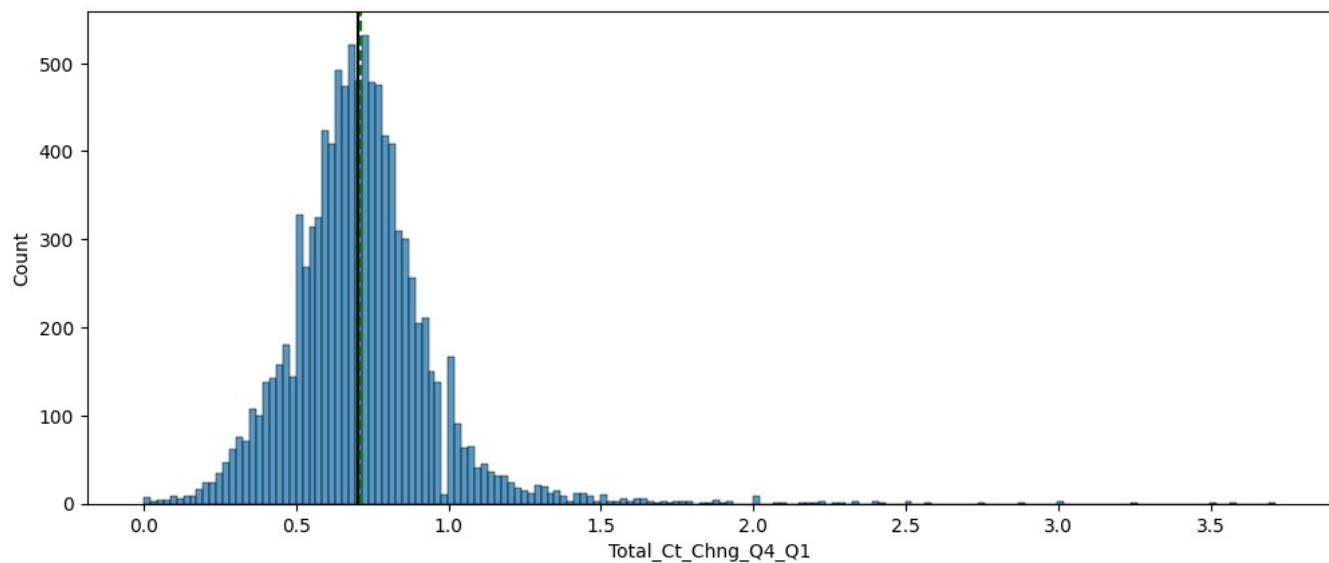
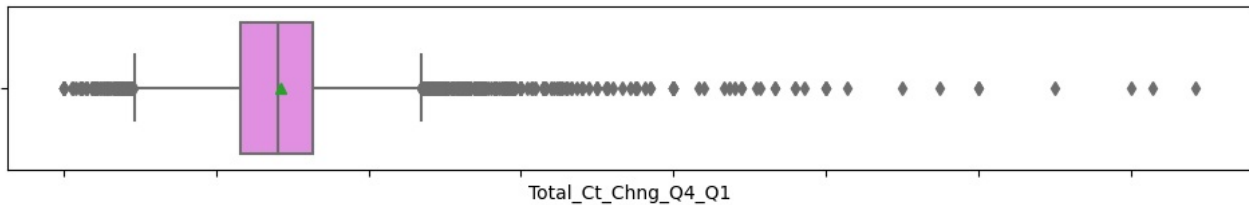


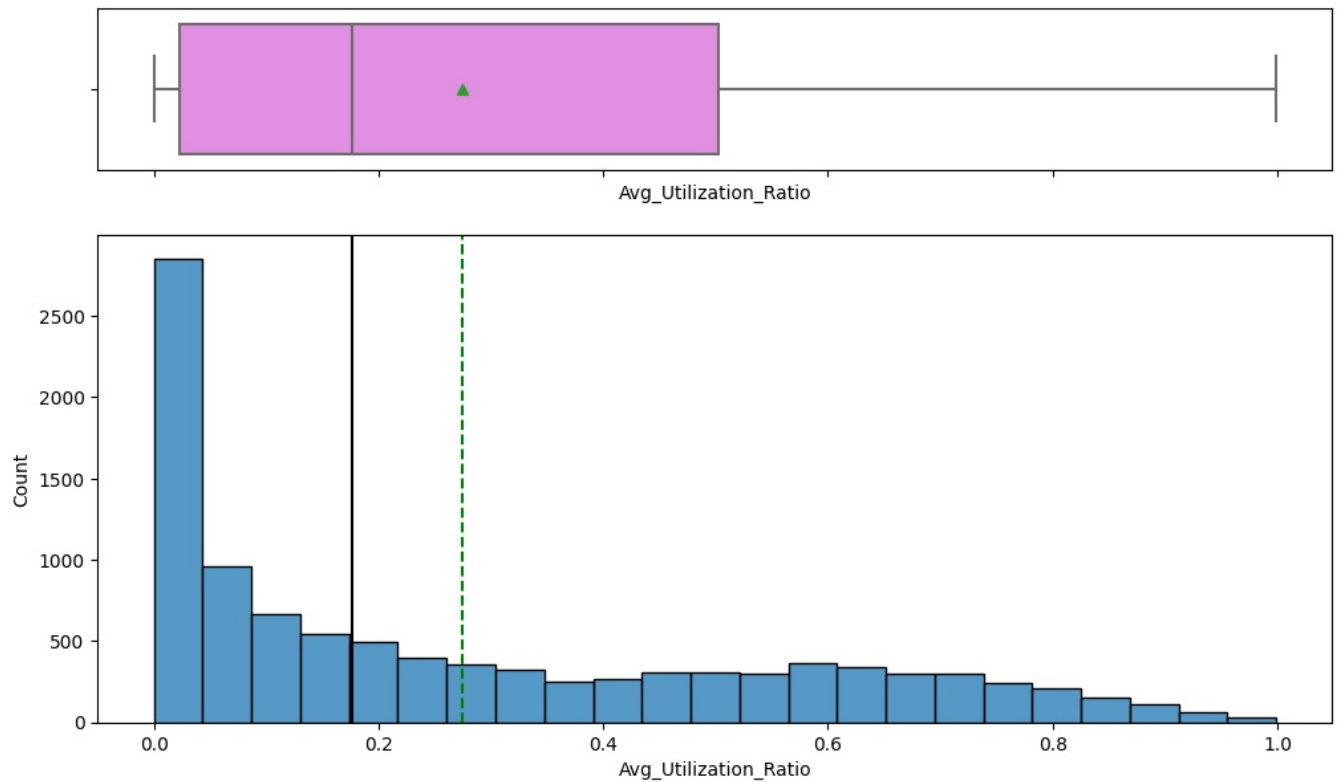












- Distribution of age is normal.
- Distribution of the credit amount is right-skewed.
- Median of the amount spent on fruits lies around 37 months on the books.
- A large number of 0~100 USD customers that have very low balances.
- Distribution for the number of products per customer is left-skewed.
- There are outliers in the amount spent above 12500. We will not remove all such data points as they represent real customers.
- ~65 transactions in the last 12 months by the majority of the customers.
- The median of the distribution is ~.60

In [21]: `# function to create labeled barplots`

```
def labeled_barplot(data, feature, perc=False, n=None):
    """
    Barplot with percentage at the top

    data: dataframe
    feature: dataframe column
    perc: whether to display percentages instead of count (default is False)
    n: displays the top n category levels (default is None, i.e., display all levels)
    """

    total = len(data[feature]) # length of the column
    count = data[feature].nunique()
    if n is None:
        plt.figure(figsize=(count + 1, 5))
    else:
        plt.figure(figsize=(n + 1, 5))

    plt.xticks(rotation=90, fontsize=15)
    ax = sns.countplot(
        data=data,
        x=feature,
        palette="Paired",
        order=data[feature].value_counts().index[:n].sort_values(),
    )

    for p in ax.patches:
        if perc == True:
            label = "{:.1f}%".format(
                100 * p.get_height() / total
            ) # percentage of each class of the category
        else:
            label = p.get_height() # count of each level of the category

        x = p.get_x() + p.get_width() / 2 # width of the plot
        y = p.get_height() # height of the plot

        ax.annotate(
            label,
            (x, y),
```

```

        ha="center",
        va="center",
        size=12,
        xytext=(0, 5),
        textcoords="offset points",
    ) # annotate the percentage

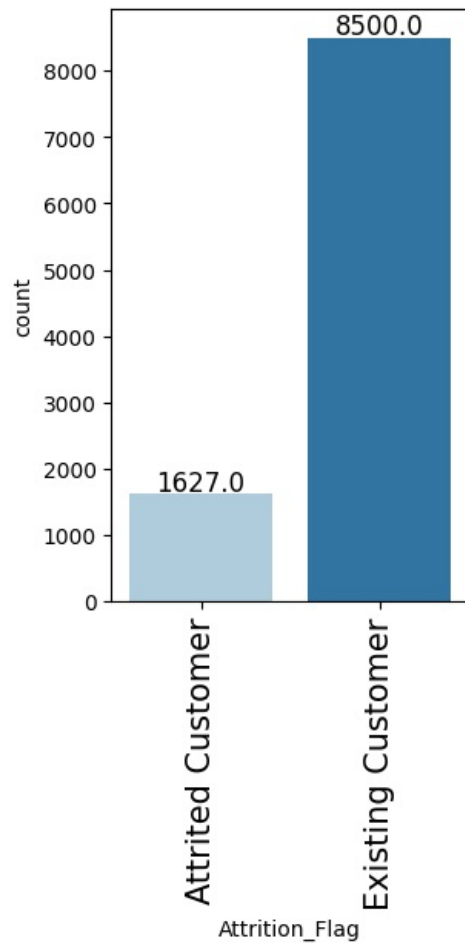
plt.show() # show the plot

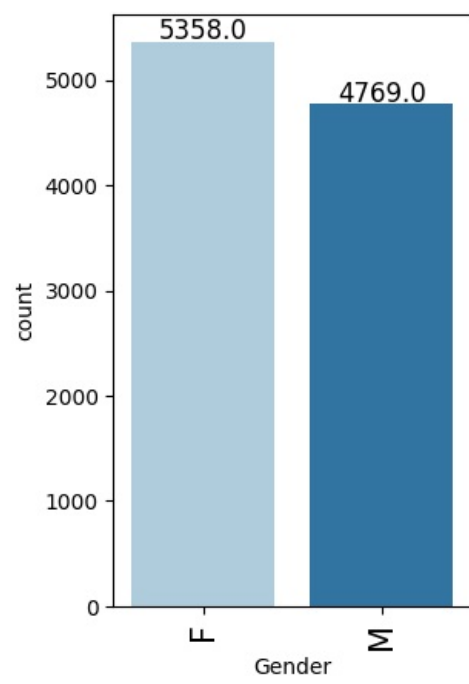
```

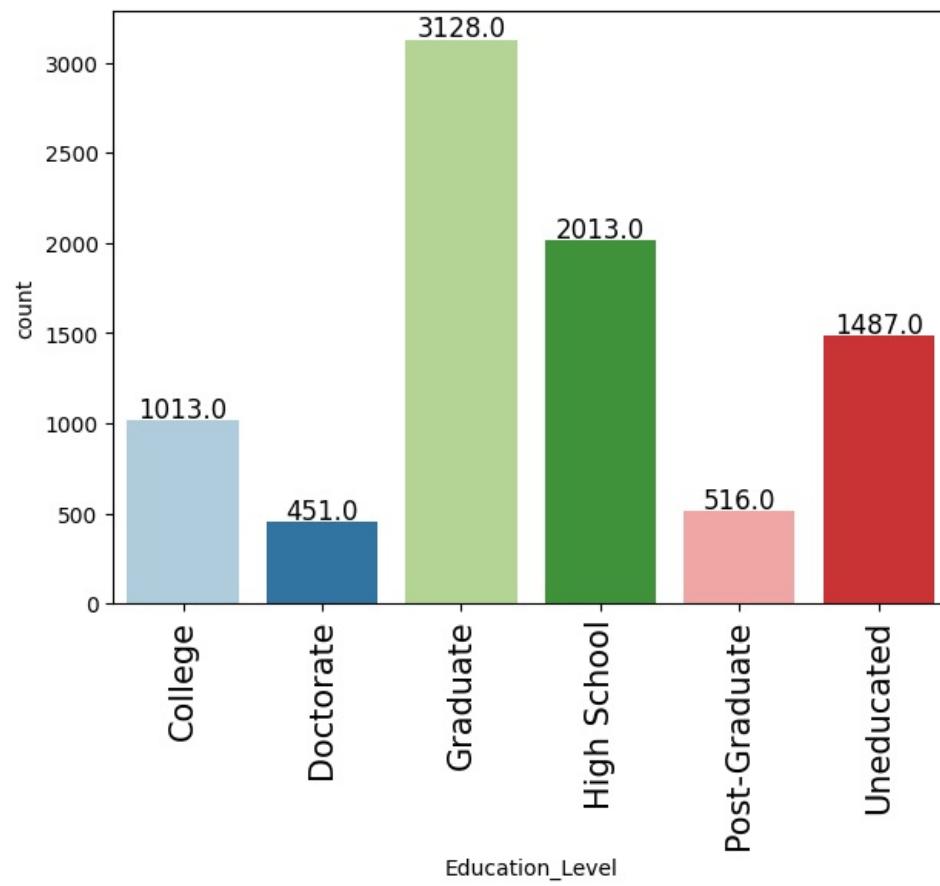
```

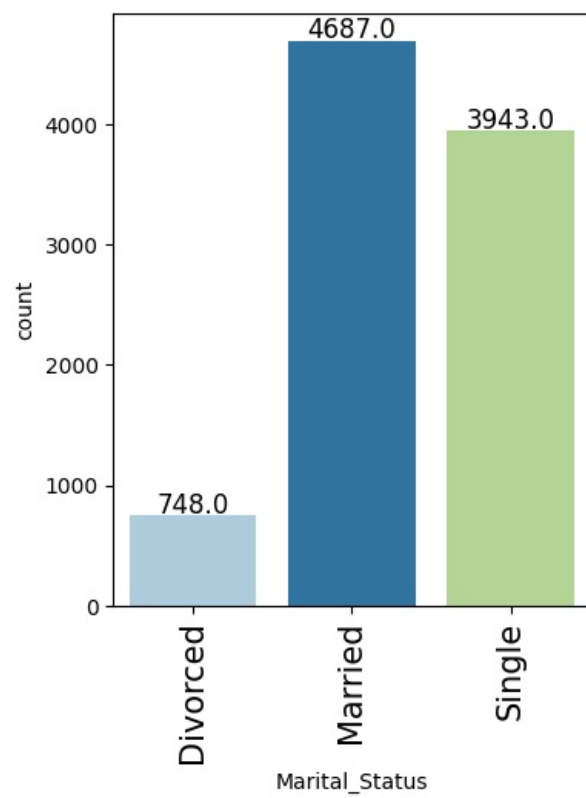
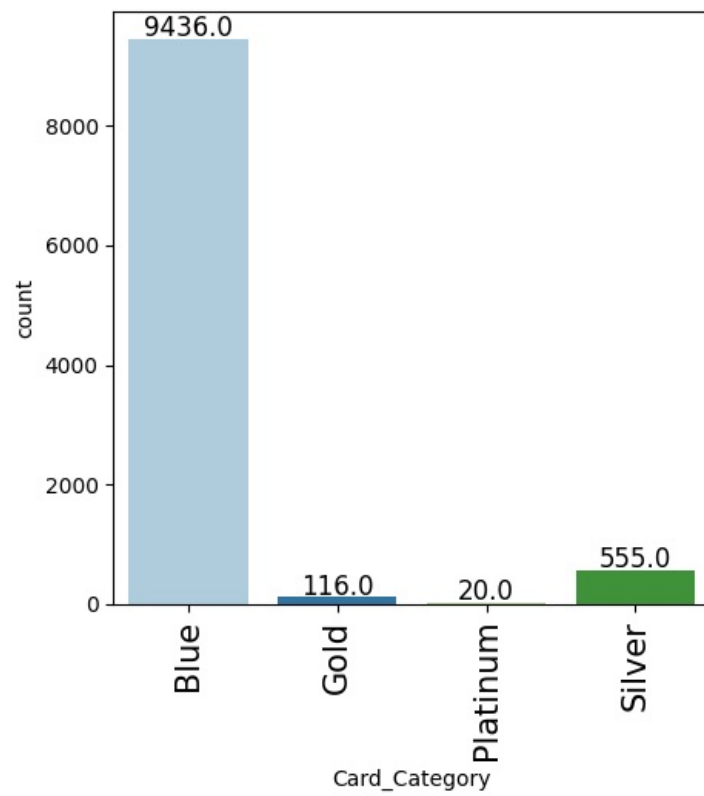
In [22]: # observations on Attrition_Flag
labeled_barplot(data, "Attrition_Flag")
# observations on Sex
labeled_barplot(data, "Gender")
# observations on barplot
labeled_barplot(data, "Education_Level")
# observations on barplot
labeled_barplot(data, "Card_Category")
# observations on Saving accounts
labeled_barplot(data, "Marital_Status")
# observations on barplot
labeled_barplot(data, "Contacts_Count_12_mon")

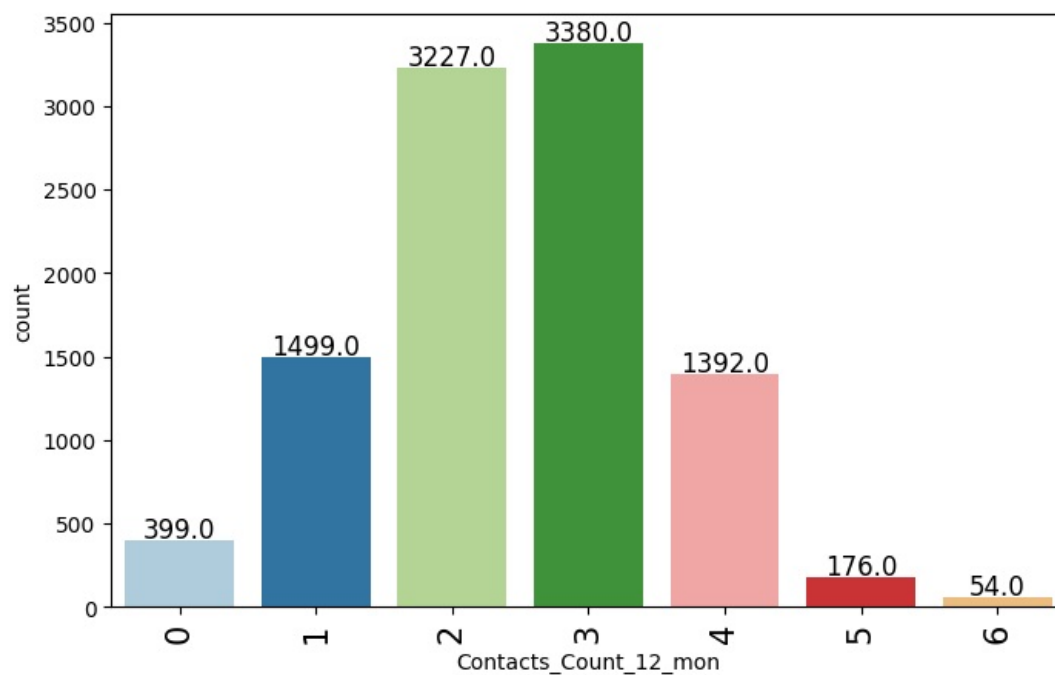
```











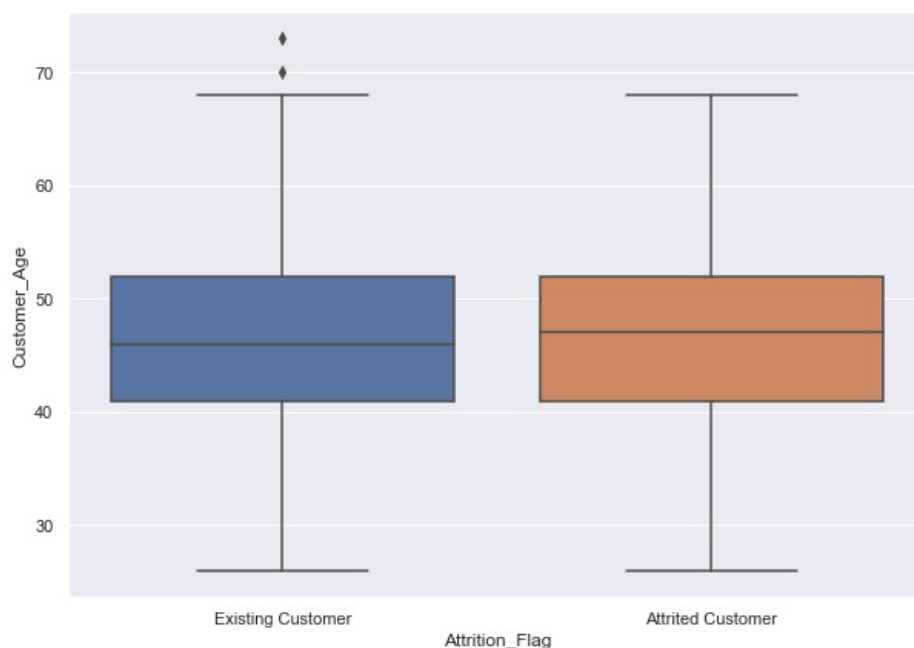
- 83% observations for Existing Customers and 16% observations for Attritioners.
- 47% are male customers and 53% are female customers.
- 19% of customers are high school graduates.
- 14% of customers have no formal education and may not qualify for a credit card.
- 94% fall into the Blue category.

Bivariate Analysis

Let's explore this further with the help of other plots.

```
In [ ]: # box plots on 2 variables
sns.set(rc={"figure.figsize": (10, 7)})
sns.boxplot(x="Attrition_Flag", y="Customer_Age", data=data, orient="vertical")
```

```
Out[ ]: <AxesSubplot:xlabel='Attrition_Flag', ylabel='Customer_Age'>
```

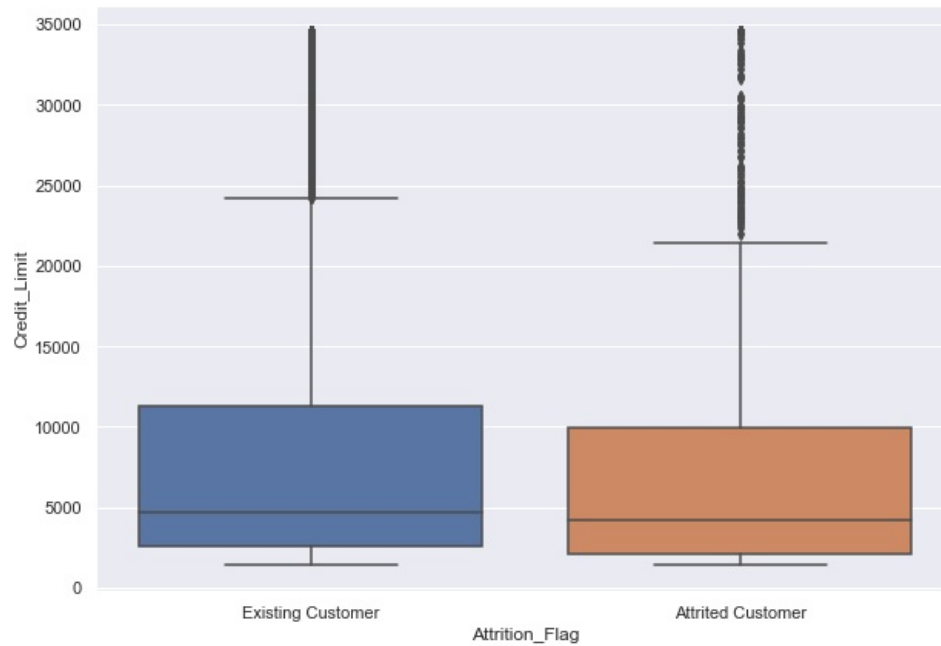


- the median age of attritees is higher than the median age of non-attritees.
- This shows that younger customers are more likely to stay as customers.
- There are outliers in boxplots of only existing customers.

```
In [ ]: sns.set(rc={"figure.figsize": (10, 7)})
```

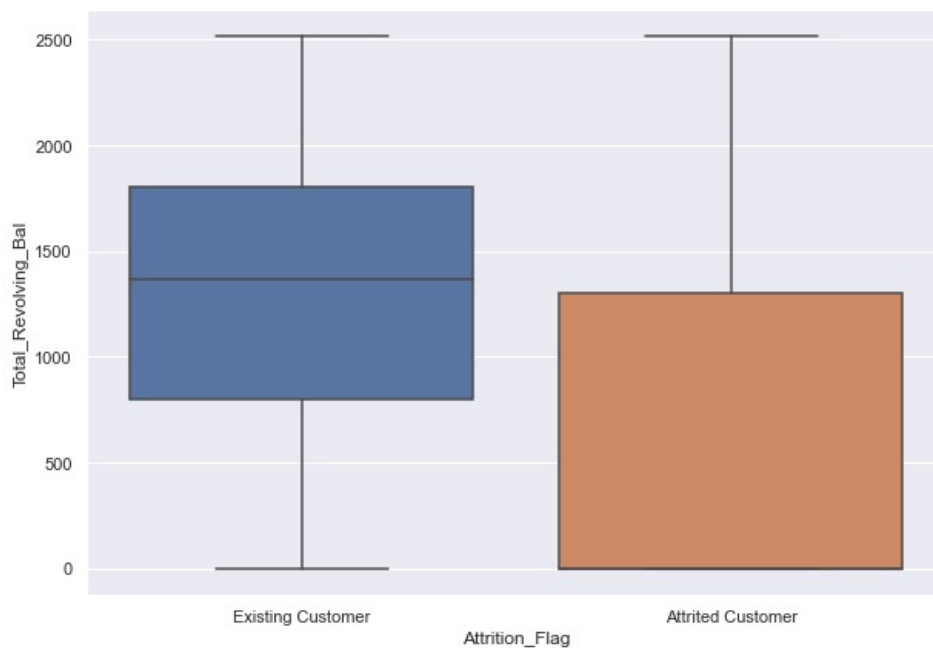
```
sns.boxplot(x="Attrition_Flag", y="Credit_Limit", data=data, orient="vertical")
```

```
Out[ ]: <AxesSubplot:xlabel='Attrition_Flag', ylabel='Credit_Limit'>
```



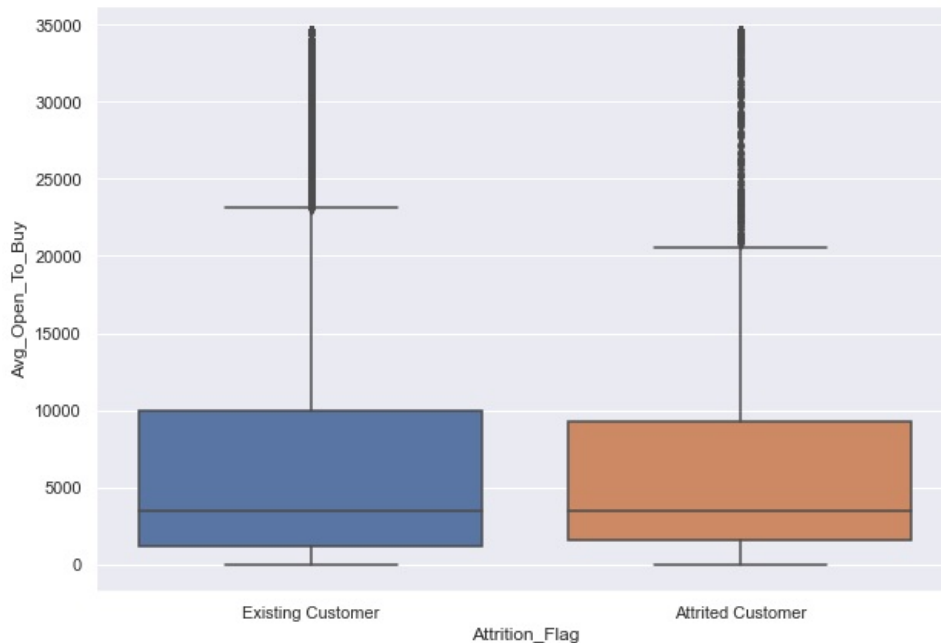
```
In [ ]: sns.set(rc={"figure.figsize": (10, 7)})  
sns.boxplot(x="Attrition_Flag", y="Total_Revolving_Bal", data=data, orient="vertical")
```

```
Out[ ]: <AxesSubplot:xlabel='Attrition_Flag', ylabel='Total_Revolving_Bal'>
```



```
In [ ]: sns.set(rc={"figure.figsize": (10, 7)})  
sns.boxplot(x="Attrition_Flag", y="Avg_Open_To_Buy", data=data)
```

```
Out[ ]: <AxesSubplot:xlabel='Attrition_Flag', ylabel='Avg_Open_To_Buy'>
```



- Customers with higher amounts left on the credit card tend to be existing customers.

```
In [ ]: # function to plot stacked bar chart

def stacked_barplot(data, predictor, target):
    """
    Print the category counts and plot a stacked bar chart

    data: dataframe
    predictor: independent variable
    target: target variable
    """
    count = data[predictor].nunique()
    sorter = data[target].value_counts().index[-1]
    tab1 = pd.crosstab(data[predictor], data[target], margins=True).sort_values(
        by=sorter, ascending=False
    )
    print(tab1)
    print("-" * 120)
    tab = pd.crosstab(data[predictor], data[target], normalize="index").sort_values(
        by=sorter, ascending=False
    )
    tab.plot(kind="bar", stacked=True, figsize=(count + 1, 5))
    plt.legend(
        loc="lower left", frameon=False,
    )
    plt.legend(loc="upper left", bbox_to_anchor=(1, 1))
    plt.show()
```

```
In [ ]: ### Function to plot distributions

def distribution_plot_wrt_target(data, predictor, target):

    fig, axs = plt.subplots(2, 2, figsize=(12, 10))

    target_uniq = data[target].unique()

    axs[0, 0].set_title("Distribution of target for target=" + str(target_uniq[0]))
```

```

sns.histplot(
    data=data[data[target] == target_uniq[0]],
    x=predictor,
    kde=True,
    ax=axes[0, 0],
    color="teal",
)

axes[0, 1].set_title("Distribution of target for target=" + str(target_uniq[1]))
sns.histplot(
    data=data[data[target] == target_uniq[1]],
    x=predictor,
    kde=True,
    ax=axes[0, 1],
    color="orange",
)

axes[1, 0].set_title("Boxplot w.r.t target")
sns.boxplot(data=data, x=target, y=predictor, ax=axes[1, 0], palette="gist_rainbow")

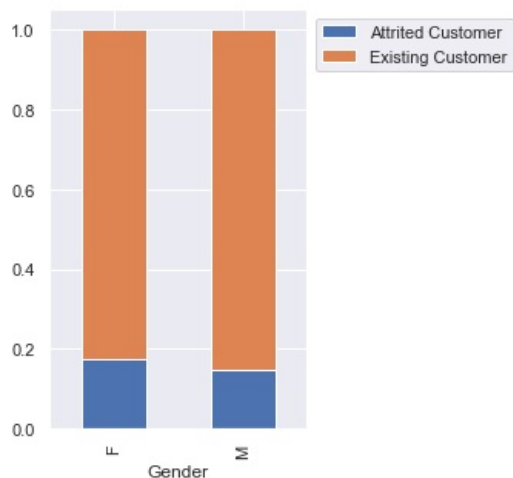
axes[1, 1].set_title("Boxplot (without outliers) w.r.t target")
sns.boxplot(
    data=data,
    x=target,
    y=predictor,
    ax=axes[1, 1],
    showfliers=False,
    palette="gist_rainbow",
)

plt.tight_layout()
plt.show()

```

```
In [ ]: stacked_barplot(data, "Gender", "Attrition_Flag")
```

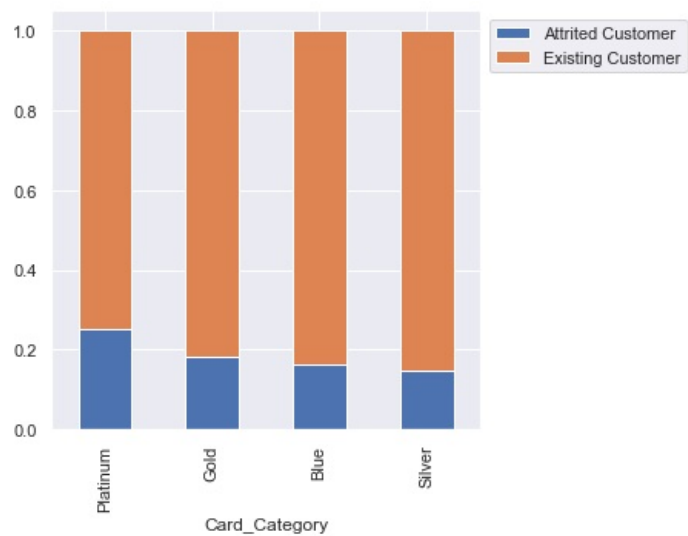
Attrition_Flag	Attrited Customer	Existing Customer	All
Gender			
All	1627	8500	10127
F	930	4428	5358
M	697	4072	4769



- The dataset is skewed towards females.

```
In [ ]: stacked_barplot(data, "Card_Category", "Attrition_Flag")
```

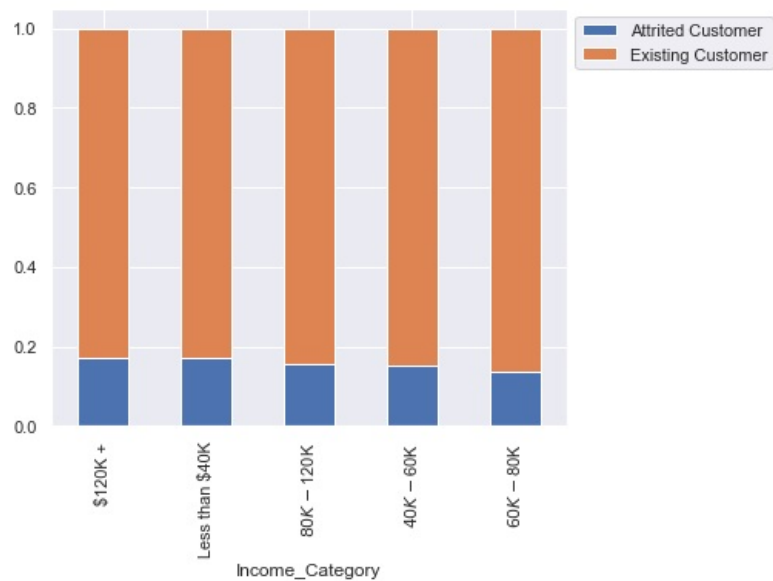
Attrition_Flag	Attrited Customer	Existing Customer	All
Card_Category			
All	1627	8500	10127
Blue	1519	7917	9436
Silver	82	473	555
Gold	21	95	116
Platinum	5	15	20



- Customers having Platinum or Gold cards are more likely to attrit.
- Customers having Silver or Blue cards are less likely to attrit.

```
In [ ]: stacked_barplot(data, "Income_Category", "Attrition_Flag")
```

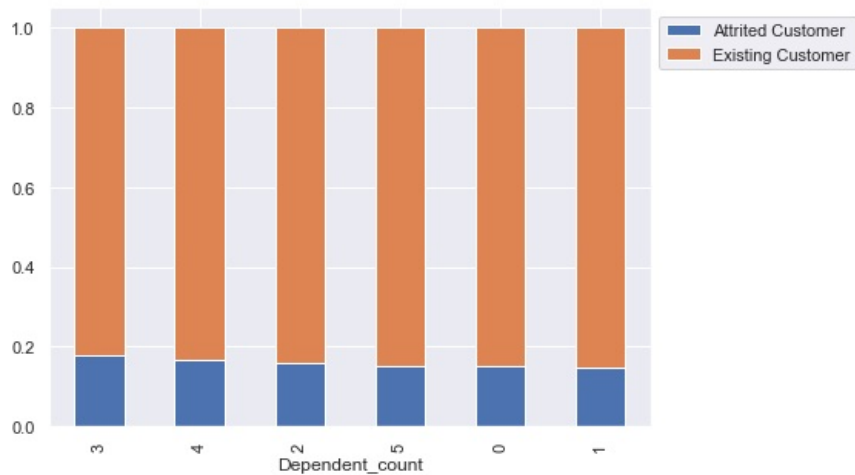
Attrition_Flag	Attrited Customer	Existing Customer	All
Income_Category			
All	1627	8500	10127
Less than \$40K	799	3874	4673
\$40K - \$60K	271	1519	1790
\$80K - \$120K	242	1293	1535
\$60K - \$80K	189	1213	1402
\$120K +	126	601	727



- The ratio amongst all income levels is more or less equal.

```
In [ ]: stacked_barplot(data, "Dependent_count", "Attrition_Flag")
```

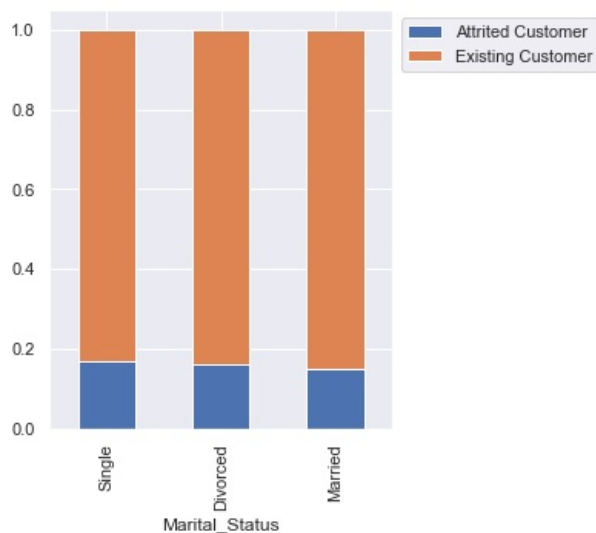
Attrition_Flag	Attrited Customer	Existing Customer	All
Dependent_count			
All	1627	8500	10127
3	482	2250	2732
2	417	2238	2655
1	269	1569	1838
4	260	1314	1574
0	135	769	904
5	64	360	424



- Customers having 3 or 4 dependents have a slightly higher chance of attrition than others.
- There is little difference in the ratio of existing vs attrited customers per number of dependents.

```
In [ ]: stacked_barplot(data, "Marital_Status", "Attrition_Flag")
```

Attrition_Flag	Attrited Customer	Existing Customer	All
Marital_Status			
All	1498	7880	9378
Married	709	3978	4687
Single	668	3275	3943
Divorced	121	627	748



- Customers who are single have a higher attrition rate than non-singles.
- Married people have most attritees.

```
In [ ]: # Correlation matrix
plt.figure(figsize=(15, 7))
sns.heatmap(data.corr(), annot=True, vmin=-1, vmax=1, fmt=".2f", cmap="Spectral")
plt.show()
```



- Total_Trans_Ct is highly correlated with Total_Trans_Amt which is 0.81.
- Months_on_book is highly correlated to Customer_Age.
- Avg_Utilization_Ratio has a correlation with Total_Revolving_Bal which is 0.62.

Data Preparation for Modeling

```
In [ ]: # label encode the target variable
data.Attrition_Flag = data.Attrition_Flag.replace(
    to_replace={"Attrited Customer": 1, "Existing Customer": 0}
)
data.Attrition_Flag
```

```
Out[ ]: 0      0
        1      0
        2      0
        3      0
        4      0
        ..
10122    0
10123    1
10124    1
10125    1
10126    1
Name: Attrition_Flag, Length: 10127, dtype: int64
```

```
In [ ]: # print head
data.head()
```

```
Out[ ]:  Attrition_Flag  Customer_Age  Gender  Dependent_count  Education_Level  Marital_Status  Income_Category  Card_Category  Months_on_bo
0              0           45      M           3      High School      Married      60K - 80K      Blue
1              0           49      F           5      Graduate      Single      Less than $40K      Blue
2              0           51      M           3      Graduate      Married      80K - 120K      Blue
3              0           40      F           4      High School      NaN      Less than $40K      Blue
4              0           40      M           3      Uneducated      Married      60K - 80K      Blue
```

```
In [ ]: # Separate target variable and other variables
X = data.drop(columns="Attrition_Flag")

Y = data["Attrition_Flag"]

# check head
X.head()
```

	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category	Months_on_book	Total_Relat
0	45	M	3	High School	Married	60K – 80K	Blue	39	
1	49	F	5	Graduate	Single	Less than \$40K	Blue	44	
2	51	M	3	Graduate	Married	80K – 120K	Blue	36	
3	40	F	4	High School	NaN	Less than \$40K	Blue	34	
4	40	M	3	Uneducated	Married	60K – 80K	Blue	21	

```
In [ ]: # Dummy variables
X = pd.get_dummies(
    data=data,
    columns=[
        "Gender",
        "Education_Level", # has missing values
        "Marital_Status", # has missing values
        "Income_Category",
        "Card_Category",
    ],
    drop_first=True,
)
```

```
In [ ]: # drop target variable
X = X.drop(columns="Attrition_Flag")
```

```
In [ ]: # check head
X.head()
```

	Customer_Age	Dependent_count	Months_on_book	Total_Relationship_Count	Months_Inactive_12_mon	Contacts_Count_12_mon	Credit_Li
0	45	3	39	5	1	3	12691.0
1	49	5	44	6	1	2	8256.0
2	51	3	36	4	1	0	3418.0
3	40	4	34	3	4	1	3313.0
4	40	3	21	5	1	0	4716.0

```
In [ ]: # print percentage of each unique value in the Y series
Y.value_counts() / 100
```

```
Out[ ]: 0    85.000
        1    16.270
        Name: Attrition_Flag, dtype: float64
```

```
In [ ]: X_temp, X_test, y_temp, y_test = train_test_split(
        X, Y, test_size=0.20, random_state=1, stratify=Y
    )

    # Split the temporary set into train and validation

    X_train, X_val, y_train, y_val = train_test_split(
        X_temp,
        y_temp,
        test_size=0.25,
        random_state=1,
        stratify=y_temp, # set the weighting feature on
    )
    print(X_train.shape, X_val.shape, X_test.shape)

(6075, 29) (2026, 29) (2026, 29)
```

```
In [ ]: # check head
X_train.head()
```

	Customer_Age	Dependent_count	Months_on_book	Total_Relationship_Count	Months_Inactive_12_mon	Contacts_Count_12_mon	Credit
800	40	2	21	6	4	3	200
498	44	1	34	6	2	0	28
4356	48	4	36	5	1	2	67
407	41	2	36	6	2	0	270
8728	46	4	36	2	2	3	150

Missing-Value Treatment

- We will use median to impute missing values in `Education_Level`, `Marital_Status`.

```
In [ ]: # show missing values
data.isna().sum()
```

```
Out[ ]: Attrition_Flag          0
Customer_Age              0
Gender                    0
Dependent_count           0
Education_Level          1519
Marital_Status            749
Income_Category           0
Card_Category             0
Months_on_book            0
Total_Relationship_Count  0
Months_Inactive_12_mon    0
Contacts_Count_12_mon     0
Credit_Limit              0
Total_Revolving_Bal       0
Avg_Open_To_Buy           0
Total_Amt_Chng_Q4_Q1      0
Total_Trans_Amt           0
Total_Trans_Ct            0
Total_Ct_Chng_Q4_Q1       0
Avg_Utilization_Ratio     0
dtype: int64
```

```
In [ ]: # view missing data for type
data[data.Education_Level.isnull()]
```

```
Out[ ]:   Attrition_Flag  Customer_Age  Gender  Dependent_count  Education_Level  Marital_Status  Income_Category  Card_Category  Months_o
```

	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category	Months_o
6	0	51	M	4	NaN	Married	\$120K +	Gold	
11	0	65	M	1	NaN	Married	40K – 60K	Blue	
15	0	44	M	4	NaN	NaN	80K – 120K	Blue	
17	0	41	M	3	NaN	Married	80K – 120K	Blue	
23	0	47	F	4	NaN	Single	Less than \$40K	Blue	
...	...	...	...	...	...	...	...	...	...
10090	0	36	F	3	NaN	Married	40K – 60K	Blue	
10094	0	59	M	1	NaN	Single	60K – 80K	Blue	
10095	0	46	M	3	NaN	Married	80K – 120K	Blue	
10118	1	50	M	1	NaN	NaN	80K – 120K	Blue	
10123	1	41	M	2	NaN	Divorced	40K – 60K	Blue	

1519 rows × 20 columns

```
In [ ]: # Let's impute the missing values
imputer = SimpleImputer(missing_values=np.nan, strategy="most_frequent")
impute = imputer.fit(X_train)

# impute on train, validation set and test set
X_train = impute.transform(X_train)
X_val = imputer.transform(X_val)
X_test = imputer.transform(X_test)
```

```
In [ ]: # check for missingness in training set
np.nan in X_train
```

```
Out[ ]: False
```

```
In [ ]: # get column count
len(X_train.T)
```

```
Out[ ]: 29
```

Model Building

Model evaluation criterion

The nature of predictions made by the classification model will translate as follows:

- True positives (TP) are failures correctly predicted by the model.
- False negatives (FN) are real failures in a generator where there is no detection by model.
- False positives (FP) are failure detections in a generator where there is no failure.

Which metric to optimize?

- We need to choose the metric which will ensure that the maximum number of generator failures are predicted correctly by the model.
- We would want Recall to be maximized as greater the Recall, the higher the chances of minimizing false negatives.
- We want to minimize false negatives because if a model predicts that a machine will have no failure when there will be a failure, it will increase the maintenance cost.

Let's define a function to output different metrics (including recall) on the train and test set and a function to show confusion matrix so that we do not have to use the same code repetitively while evaluating models.

Missing value imputation

```
In [ ]: models = [] # Empty list to store all the models

# Appending models into the list
models.append(("Bagging", BaggingClassifier(random_state=1)))
models.append(("Random forest", RandomForestClassifier(random_state=1)))
# Complete the code to append remaining 3 models in the list models

print("\n" "Training Performance:" "\n")
for name, model in models:
    model.fit(X_train, y_train)
    scores = recall_score(y_train, model.predict(X_train))
    print("{}: {}".format(name, scores))

print("\n" "Validation Performance:" "\n")

for name, model in models:
    model.fit(X_train, y_train)
    scores_val = recall_score(y_val, model.predict(X_val))
    print("{}: {}".format(name, scores_val))
```

Cross-Validation Performance:

LogisticRegression: 81.65724751439036
DecisionTree: 74.6902145473574
GBM: 81.24646781789639
Adaboost: 81.3469387755102
Xgboost: 86.37100994243852
Bagging: 78.48142333856619

Validation Performance:

LogisticRegression: 0.8098159509202454
DecisionTree: 0.803680981595092
GBM: 0.8588957055214724
Adaboost: 0.8588957055214724
Xgboost: 0.8803680981595092
Bagging: 0.7975460122699386

Model Building with Undersampled data

```
In [ ]: # Random undersampler for under sampling the data
rus = RandomUnderSampler(random_state=1, sampling_strategy=1)
X_train_un, y_train_un = rus.fit_resample(X_train, y_train)
```

Model Building with Oversampled data

```
In [ ]: # Synthetic Minority Over Sampling Technique
sm = SMOTE(sampling_strategy=1, k_neighbors=5, random_state=1)
X_train_over, y_train_over = sm.fit_resample(X_train, y_train)
```

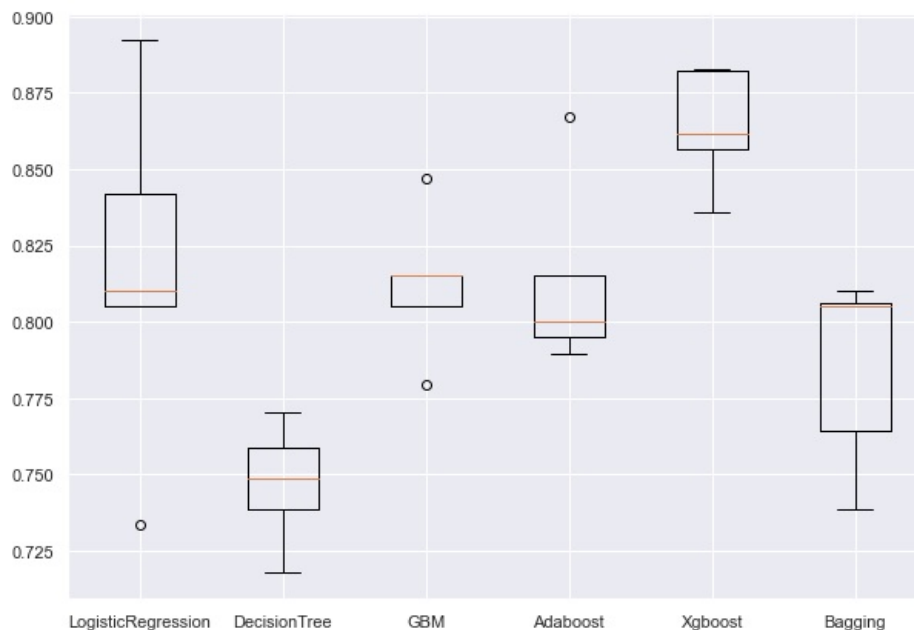
```
In [ ]: # Plotting boxplots for CV scores of all models defined above
fig = plt.figure(figsize=(10, 7))

fig.suptitle("CV Comparison")
ax = fig.add_subplot(111)

plt.boxplot(results)
ax.set_xticklabels(names)

plt.show()
```

CV Comparison



- We can see that XGBoost is giving the highest cross-validated recall followed by Ada/GradientBoost.
- The boxplot shows that the performance of AdaBoost and GradientBoost are consistent.
- The Performance of Adaboost and XGBoost is generalised on validation set as well.
- We will tune - Adaboost, GradientBoost and XGBoost and see if the performance improves.

Hyperparameter Tuning

We will tune Adaboost, GradientBoost and xgboost models using GridSearchCV and RandomizedSearchCV. We will also compare the performance and time taken by these two methods - grid search and randomized search.

First, let's create two functions to calculate different metrics and confusion matrix so that we don't have to use the same code repeatedly for each model.

```
In [ ]: # defining a function to compute different metrics to check performance of a classification model built using s
def model_performance_classification_sklearn(model, predictors, target):
    """
    Function to compute different metrics to check classification model performance

    model: classifier
    predictors: independent variables
    target: dependent variable
    """

    # predicting using the independent variables
    pred = model.predict(predictors)

    acc = accuracy_score(target, pred) # to compute Accuracy
    recall = recall_score(target, pred) # to compute Recall
    precision = precision_score(target, pred) # to compute Precision
    f1 = f1_score(target, pred) # to compute F1-score

    # creating a dataframe of metrics
    df_perf = pd.DataFrame(
        {"Accuracy": acc, "Recall": recall, "Precision": precision, "F1": f1},
        index=[0],
    )

    return df_perf
```

Hyperparameter tuning can take a long time to run, so to avoid that time complexity - you can use the following grids, wherever required.

- For Gradient Boosting:

```
param_grid = {
    "init": [AdaBoostClassifier(random_state=1), DecisionTreeClassifier(random_state=1)],
    "n_estimators": np.arange(75, 150, 25),
    "learning_rate": [0.1, 0.01, 0.2, 0.05, 1],
    "subsample": [0.5, 0.7, 1],
}
```

```
    "max_features": [0.5, 0.7, 1],
}
```

- For Adaboost:

```
param_grid = {
    "n_estimators": np.arange(10, 110, 10),
    "learning_rate": [0.1, 0.01, 0.2, 0.05, 1],
    DecisionTreeClassifier(max_depth=1, random_state=1),
    DecisionTreeClassifier(max_depth=2, random_state=1),
    DecisionTreeClassifier(max_depth=3, random_state=1),
}
```

- For Bagging Classifier:

```
param_grid = {
    'max_samples': [0.8, 0.9, 1],
    'max_features': [0.7, 0.8, 0.9],
    'n_estimators': [30, 50, 70],
}
```

- For Random Forest:

```
param_grid = {
    "n_estimators": [200, 250, 300],
    "min_samples_leaf": np.arange(1, 4),
    "max_features": [np.arange(0.3, 0.6, 0.1), 'sqrt'],
    "max_samples": np.arange(0.4, 0.7, 0.1)
}
```

- For Decision Trees:

```
param_grid = {
    'max_depth': np.arange(2, 6),
    'min_samples_leaf': [1, 4, 7],
    'max_leaf_nodes': [10, 15],
    'min_impurity_decrease': [0.0001, 0.001]
}
```

- For XGBoost:

```
param_grid={
    'n_estimators': np.arange(50, 300, 50),
    'scale_pos_weight': [0, 1, 2, 5, 10],
    'learning_rate': [0.01, 0.1, 0.2, 0.05],
    'gamma': [0, 1, 3, 5],
    'subsample': [0.7, 0.8, 0.9, 1]
}
```

```
In [ ]: def confusion_matrix_sklearn(model, predictors, target):
    """
    To plot the confusion_matrix with percentages

    model: classifier
    predictors: independent variables
    target: dependent variable
    """
    y_pred = model.predict(predictors)
    cm = confusion_matrix(target, y_pred)
    labels = np.asarray(
        [
            ["{0:0.0f}".format(item) + "\n{0:.2%}".format(item / cm.flatten().sum())]
            for item in cm.flatten()
        ]
    ).reshape(2, 2)

    # plot the matrix inside a heatmap
    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=labels, fmt="")
    plt.ylabel("True label")
    plt.xlabel("Predicted label")
```

Sample tuning method for Decision tree with oversampled data

GridSearchCV

```
In [ ]: %%time

# defining model
model = GradientBoostingClassifier(random_state=1)

# Parameter grid to pass in GridSearchCV
param_grid = {
    "n_estimators": np.arange(10, 110, 10),
    "learning_rate": [0.1, 0.01, 0.2, 0.05, 1]
}

# Type of scoring used to compare parameter combinations
scorer = metrics.make_scorer(metrics.recall_score)

# Calling GridSearchCV
grid_cv = GridSearchCV(estimator=model, param_grid=param_grid, scoring=scorer, cv=5, n_jobs = -1)

# Fitting parameters in GridSeachCV
grid_cv.fit(X_train, y_train)

print(
    "Best Parameters:{} \nScore: {}".format(grid_cv.best_params_, grid_cv.best_score_)
)

Best Parameters: {'learning_rate': 0.2, 'n_estimators': 100}
Score: 0.8462899005756149
Wall time: 1min 3s
```

```
In [ ]: # extract best estimator from the model
gbgscv = grid_cv.best_estimator_
```

```
In [ ]: # fit the model
gbgscv.fit(X_train, y_train)
```

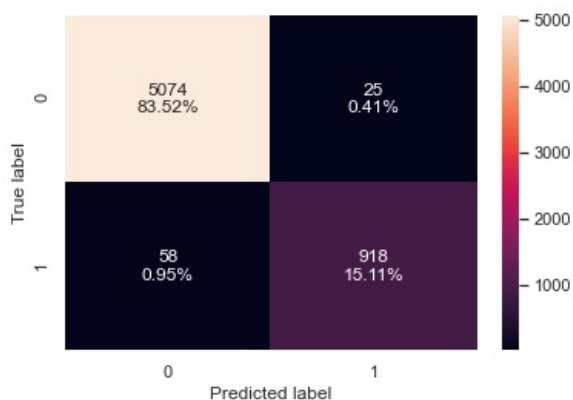
```
Out[ ]: GradientBoostingClassifier(learning_rate=0.2, random_state=1)
```

```
In [ ]: # Calculating different metrics on train set
print("Training performance:")
gradient_grid_train = model_performance_classification_sklearn(gbgscv, X_train, y_train)
display(gradient_grid_train)
```

Training performance:

	Accuracy	Recall	Precision	F1
0	0.986	0.941	0.973	0.957

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(gbgscv, X_train, y_train)
```

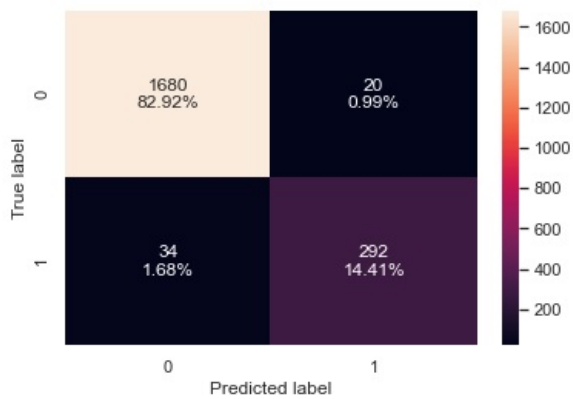


```
In [ ]: # Calculating different metrics on validation set
print("Validation performance:")
gradient_grid_val = model_performance_classification_sklearn(gbgscv, X_val, y_val)
display(gradient_grid_val)
```

Validation performance:

	Accuracy	Recall	Precision	F1
0	0.973	0.896	0.936	0.915

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(gbgscv, X_val, y_val)
```



- The validation recall is ~90%.
- The tuned GradientBoost model is not overfitting the training data.
- The False Negative rate is ~2%.

RandomizedSearchCV

```
In [ ]: %%time

# defining model
model = GradientBoostingClassifier(random_state=1)

# Parameter grid to pass in GridSearchCV
param_grid = {
    "n_estimators": np.arange(10, 110, 10),
    "learning_rate": [0.1, 0.01, 0.2, 0.05, 1]
}

# Type of scoring used to compare parameter combinations
scorer = metrics.make_scorer(metrics.recall_score)

#Calling RandomizedSearchCV
randomized_cv = RandomizedSearchCV(estimator=model, param_distributions=param_grid, n_jobs = -1, n_iter=50, sco

#Fitting parameters in RandomizedSearchCV
randomized_cv.fit(X_train,y_train)

print("Best parameters are {} with CV score={}".format(randomized_cv.best_params_,randomized_cv.best_score_))

Best parameters are {'n_estimators': 80, 'learning_rate': 0.2} with CV score=0.8503948403334906:
Wall time: 31.2 s
```

```
In [ ]: # extract best estimator from the model
gbrs = randomized_cv.best_estimator_
```

```
In [ ]: # Fit the model on training data
gbrs.fit(X_train, y_train)
```

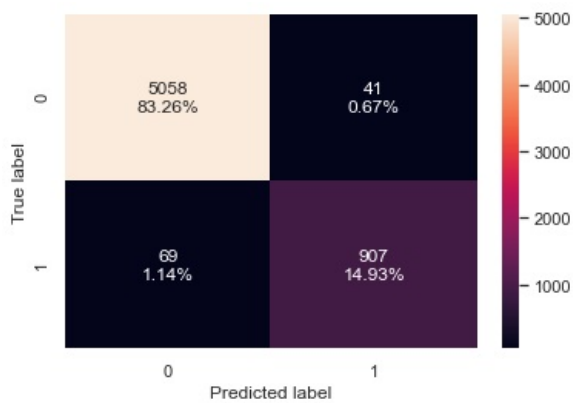
```
Out[ ]: GradientBoostingClassifier(learning_rate=0.2, n_estimators=80, random_state=1)
```

```
In [ ]: # Calculating different metrics on train set
print("Training performance:")
gradient_random_train = model_performance_classification_sklearn(gbrs, X_train, y_train)
display(gradient_random_train)
```

Training performance:

	Accuracy	Recall	Precision	F1
0	0.982	0.929	0.957	0.943

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(gbrs, X_train, y_train)
```

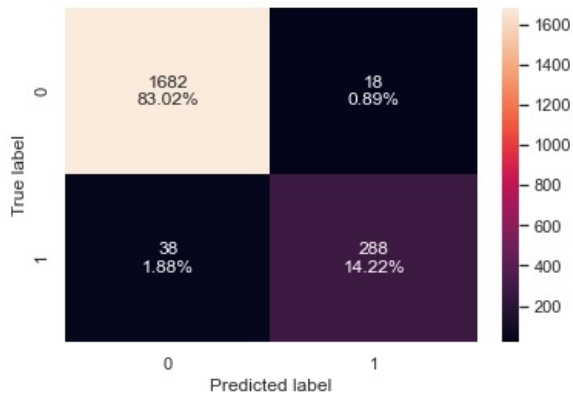


```
In [ ]: # Calculating different metrics on validation set
print("Validation performance:")
gradient_random_val = model_performance_classification_sklearn(gbrs, X_val, y_val)
display(gradient_random_val)
```

Validation performance:

	Accuracy	Recall	Precision	F1
0	0.972	0.883	0.941	0.911

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(gbrs, X_val, y_val)
```



- RandomSearchCV tuned model is not overfitting the training data.
- The False Negatives is ~2% but higher than the GradientSearchCV version.

Adaboost

GridSearchCV

```
In [ ]: %%time

# defining model
model = AdaBoostClassifier(random_state=1)

# Parameter grid to pass in GridSearchCV
param_grid = {
    "n_estimators": np.arange(10, 110, 10),
    "learning_rate": [0.1, 0.01, 0.2, 0.05, 1],
    "base_estimator": [
        DecisionTreeClassifier(max_depth=1, random_state=1),
        DecisionTreeClassifier(max_depth=2, random_state=1),
        DecisionTreeClassifier(max_depth=3, random_state=1),
    ],
}

# Type of scoring used to compare parameter combinations
scorer = metrics.make_scorer(metrics.recall_score)

# Calling GridSearchCV
grid_cv = GridSearchCV(estimator=model, param_grid=param_grid, scoring=scorer, cv=3, n_jobs = -1)

# Fitting parameters in GridSearchCV
grid_cv.fit(X_train, y_train)

print(
```

```
) "Best Parameters:{} \nScore: {}".format(grid_cv.best_params_, grid_cv.best_score_)
```

```
Best Parameters: {'base_estimator': DecisionTreeClassifier(max_depth=2, random_state=1), 'learning_rate': 1, 'n_estimators': 60}  
Score: 0.8555356300141576  
Wall time: 1min 53s
```

```
In [ ]: # get best model  
adb_tuned1 = grid_cv.best_estimator_
```

```
In [ ]: # fit the model  
adb_tuned1.fit(X_train, y_train)
```

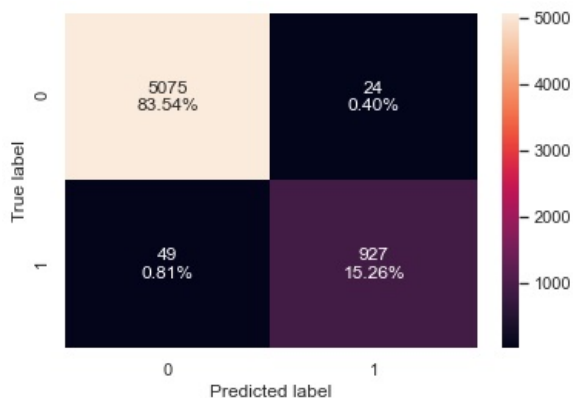
```
Out[ ]: AdaBoostClassifier(base_estimator=DecisionTreeClassifier(max_depth=2,  
                                                                random_state=1),  
                          learning_rate=1, n_estimators=60, random_state=1)
```

```
In [ ]: # Calculating different metrics on train set  
Adaboost_grid_train = model_performance_classification_sklearn(  
    adb_tuned1, X_train, y_train  
)  
print("Training performance:")  
Adaboost_grid_train
```

Training performance:

```
Out[ ]:   Accuracy  Recall  Precision   F1  
0      0.988   0.950     0.975  0.962
```

```
In [ ]: # creating confusion matrix  
confusion_matrix_sklearn(adb_tuned1, X_train, y_train)
```

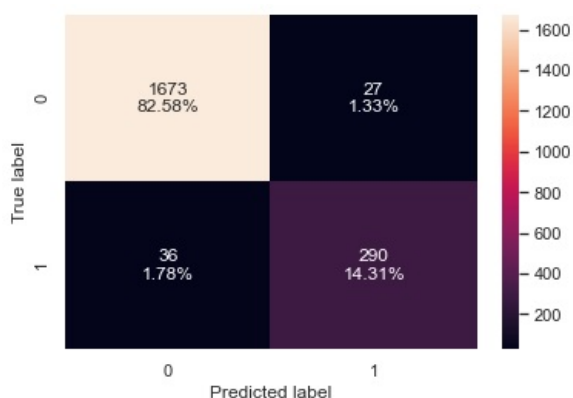


```
In [ ]: # Calculating different metrics on validation set  
Adaboost_grid_val = model_performance_classification_sklearn(adb_tuned1, X_val, y_val)  
print("Validation performance:")  
Adaboost_grid_val
```

Validation performance:

```
Out[ ]:   Accuracy  Recall  Precision   F1  
0      0.969   0.890     0.915  0.902
```

```
In [ ]: # creating confusion matrix  
confusion_matrix_sklearn(adb_tuned1, X_val, y_val)
```



Validation recall is good at about 89%

- validation recall is good at about 83%.
- The tuned Adaboost model is not overfitting the training data.

RandomizedSearchCV

```
In [ ]: %%time

# defining model
model = AdaBoostClassifier(random_state=1)

# Parameter grid to pass in GridSearchCV
param_grid = {
    "n_estimators": np.arange(10, 110, 10),
    "learning_rate": [0.1, 0.01, 0.2, 0.05, 1],
    "base_estimator": [
        DecisionTreeClassifier(max_depth=1, random_state=1),
        DecisionTreeClassifier(max_depth=2, random_state=1),
        DecisionTreeClassifier(max_depth=3, random_state=1),
    ],
}

# Type of scoring used to compare parameter combinations
scorer = metrics.make_scorer(metrics.recall_score)

#Calling RandomizedSearchCV
randomized_cv = RandomizedSearchCV(estimator=model, param_distributions=param_grid, n_jobs = -1, n_iter=50, sco

#Fitting parameters in RandomizedSearchCV
randomized_cv.fit(X_train,y_train)

print("Best parameters are {} with CV score={:}" .format(randomized_cv.best_params_,randomized_cv.best_score_))

Best parameters are {'n_estimators': 90, 'learning_rate': 1, 'base_estimator': DecisionTreeClassifier(max_depth
=2, random_state=1)} with CV score=0.8534560327198365:
Wall time: 37 s
```

```
In [ ]: # building model with best parameters
adb_tuned2 = randomized_cv.best_estimator_
# Fit the model on training data
adb_tuned2.fit(X_train, y_train)
```

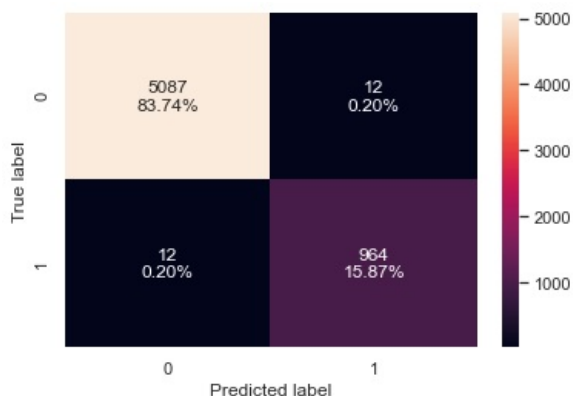
```
Out[ ]: AdaBoostClassifier(base_estimator=DecisionTreeClassifier(max_depth=2,
                                                                random_state=1),
                          learning_rate=1, n_estimators=90, random_state=1)
```

```
In [ ]: # Calculating different metrics on train set
Adaboost_random_train = model_performance_classification_sklearn(
    adb_tuned2, X_train, y_train
)
print("Training performance:")
Adaboost_random_train
```

Training performance:

```
Out[ ]:   Accuracy  Recall  Precision   F1
0      0.996  0.988      0.988  0.988
```

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(adb_tuned2, X_train, y_train)
```

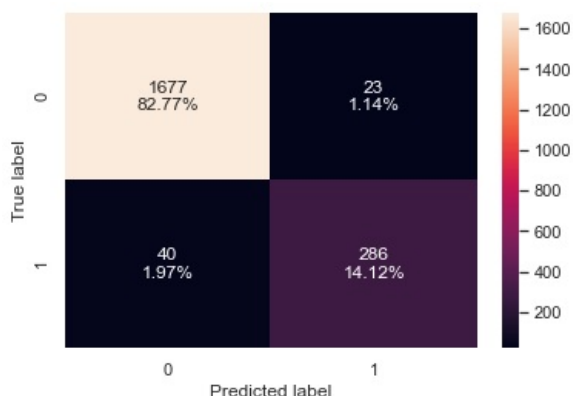


```
In [ ]: # Calculates different metrics on validation set
Adaboost_random_val = model_performance_classification_sklearn(adb_tuned2, X_val, y_val)
print("Validation performance:")
Adaboost_random_val
```

Validation performance:

```
Out[ ]: Accuracy Recall Precision F1
0 0.969 0.877 0.926 0.901
```

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(adb_tuned2, X_val, y_val)
```



- The model is not overfitting. The model in validation has a similar score to GradientBoosting model.

XGBoost

GridSearchCV

```
In [ ]: %%time

#defining model
model = XGBClassifier(random_state=1,eval_metric='logloss')

#Parameter grid to pass in GridSearchCV
param_grid={'n_estimators':np.arange(50,150,50),
            'scale_pos_weight':[2,5,10],
            'learning_rate':[0.01,0.1,0.2,0.05],
            'gamma':[0,1,3,5],
            'subsample':[0.8,0.9,1],
            'max_depth':np.arange(1,5,1),
            'reg_lambda':[5,10]}

# Type of scoring used to compare parameter combinations
scorer = metrics.make_scorer(metrics.recall_score)

#Calling GridSearchCV
grid_cv = GridSearchCV(estimator=model, param_grid=param_grid, scoring=scorer, cv=3, n_jobs = -1, verbose= 2)

#Fitting parameters in GridSeachCV
grid_cv.fit(X_train,y_train)

print("Best parameters are {} with CV score={}" .format(grid_cv.best_params_,grid_cv.best_score_))
```

Fitting 3 folds for each of 2304 candidates, totalling 6912 fits
 Best parameters are {'gamma': 0, 'learning_rate': 0.01, 'max_depth': 2, 'n_estimators': 50, 'reg_lambda': 5, 'scale_pos_weight': 10, 'subsample': 0.9} with CV score=0.9733459178857952:
 Wall time: 14min 32s

```
In [ ]: # building model with best parameters
xgb_tuned1 = grid_cv.best_estimator_

# Fit the model on training data
xgb_tuned1.fit(X_train, y_train)
```

```
Out[ ]: XGBClassifier(base_score=0.5, booster='gbtree', colsample_bylevel=1,
                    colsample_bynode=1, colsample_bytree=1, eval_metric='logloss',
                    gamma=0, gpu_id=-1, importance_type='gain',
                    interaction_constraints='', learning_rate=0.01, max_delta_step=0,
                    max_depth=2, min_child_weight=1, missing=nan,
                    monotone_constraints=('',), n_estimators=50, n_jobs=4,
                    num_parallel_tree=1, random_state=1, reg_alpha=0, reg_lambda=5,
                    scale_pos_weight=10, subsample=0.9, tree_method='exact',
                    validate_parameters=1, verbosity=None)
```

```
In [ ]: # Calculating different metrics on train set
xgboost_grid_train = model_performance_classification_sklearn(
```

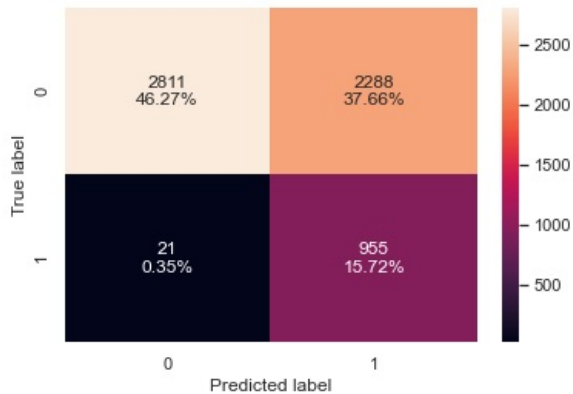
```
xgb_tuned1, X_train, y_train
)
print("Training performance:")
xgboost_grid_train
```

Training performance:

```
Out[ ]:
```

	Accuracy	Recall	Precision	F1
0	0.620	0.978	0.294	0.453

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(xgb_tuned1, X_train, y_train)
```



```
In [ ]: # Calculating different metrics on validation set
xgboost_grid_val = model_performance_classification_sklearn(xgb_tuned1, X_val, y_val)
print("Validation performance:")
xgboost_grid_val
```

Validation performance:

```
Out[ ]:
```

	Accuracy	Recall	Precision	F1
0	0.642	0.991	0.309	0.471

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(xgb_tuned1, X_val, y_val)
```



- The validation recall is higher than previous models but accuracy is too low.
- The model has not overfitted the data.

RandomizedSearchCV

```
In [ ]: %%time

# defining model
model = XGBClassifier(random_state=1, eval_metric='logloss')

# Parameter grid to pass in RandomizedSearchCV
param_grid={
    'n_estimators': np.arange(50, 150, 50),
    'scale_pos_weight': [2, 5, 10],
    'learning_rate': [0.01, 0.1, 0.2, 0.05],
    'gamma': [0, 1, 3, 5],
    'subsample': [0.8, 0.9, 1],
    'max_depth': np.arange(1, 5, 1),
    'reg_lambda': [5, 10]}

# Type of scoring used to compare parameter combinations
```

```
scorer = metrics.make_scorer(metrics.recall_score)

#Calling RandomizedSearchCV
randomized_cv = RandomizedSearchCV(estimator=model, param_distributions=param_grid, n_iter=50, scoring=scorer,

#Fitting parameters in RandomizedSearchCV
randomized_cv.fit(X_train,y_train)

print("Best parameters are {} with CV score={}" .format(randomized_cv.best_params_,randomized_cv.best_score_))
```

Best parameters are {'subsample': 1, 'scale_pos_weight': 10, 'reg_lambda': 10, 'n_estimators': 100, 'max_depth': 3, 'learning_rate': 0.05, 'gamma': 1} with CV score=0.9682208588957054:
Wall time: 17.9 s

```
In [ ]: # building model with best parameters
xgb_tuned2 = randomized_cv.best_estimator_

# Fit the model on training data
xgb_tuned2.fit(X_train, y_train)
```

```
Out[ ]: XGBClassifier(base_score=0.5, booster='gbtree', colsample_bylevel=1,
               colsample_bynode=1, colsample_bytree=1, eval_metric='logloss',
               gamma=1, gpu_id=-1, importance_type='gain',
               interaction_constraints='', learning_rate=0.05, max_delta_step=0,
               max_depth=3, min_child_weight=1, missing=nan,
               monotone_constraints='()', n_estimators=100, n_jobs=4,
               num_parallel_tree=1, random_state=1, reg_alpha=0, reg_lambda=10,
               scale_pos_weight=10, subsample=1, tree_method='exact',
               validate_parameters=1, verbosity=None)
```

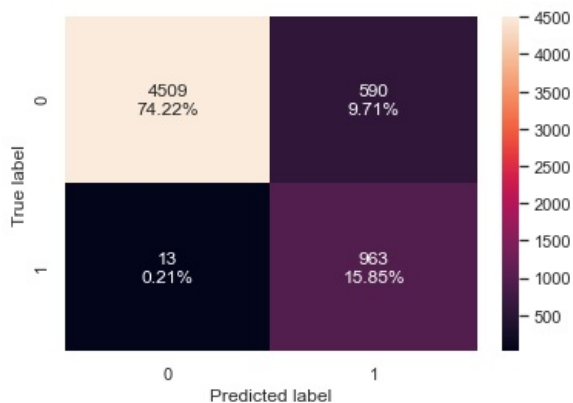
```
In [ ]: # Calculating different metrics on train set
xgboost_random_train = model_performance_classification_sklearn(
    xgb_tuned2, X_train, y_train
)
print("Training performance:")
xgboost_random_train
```

Training performance:

```
Out[ ]: 
```

	Accuracy	Recall	Precision	F1
0	0.901	0.987	0.620	0.762

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(xgb_tuned2, X_train, y_train)
```



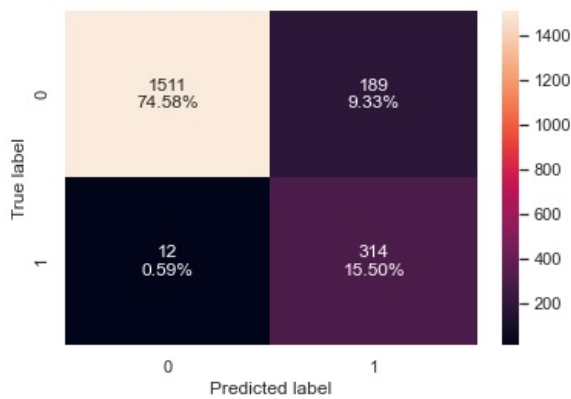
```
In [ ]: # Calculating different metrics on validation set
xgboost_random_val = model_performance_classification_sklearn(xgb_tuned2, X_val, y_val)
print("Validation performance:")
xgboost_random_val
```

Validation performance:

```
Out[ ]: 
```

	Accuracy	Recall	Precision	F1
0	0.901	0.963	0.624	0.758

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(xgb_tuned2, X_val, y_val)
```



- It is not overfitting the training and validation sets.
- False Negatives are close to 0 but higher than the GridSearchCV tuning method.

Model Comparison and Final Model Selection

Building the best modeling using oversampled data on the train and validation sets.

Training set

```
In [ ]: # print counts of target variable before oversampling
print("Before UpSampling, counts of label '1': {}".format(sum(y_train == 1)))
print("Before UpSampling, counts of label '0': {} \n".format(sum(y_train == 0)))

# Synthetic Minority Over Sampling Technique
sm = SMOTE(sampling_strategy=1, k_neighbors=5, random_state=1)

# fit and resample the training data
X_train_over, y_train_over = sm.fit_resample(X_train, y_train.ravel())

# print data size after the oversampling
print("After UpSampling, counts of label '1': {}".format(sum(y_train_over == 1)))
print("After UpSampling, counts of label '0': {} \n".format(sum(y_train_over == 0)))

print("After UpSampling, the shape of train_X: {}".format(X_train_over.shape))
print("After UpSampling, the shape of train_y: {} \n".format(y_train_over.shape))
```

Before UpSampling, counts of label '1': 976
Before UpSampling, counts of label '0': 5099

After UpSampling, counts of label '1': 5099
After UpSampling, counts of label '0': 5099

After UpSampling, the shape of train_X: (10198, 29)
After UpSampling, the shape of train_y: (10198,)

Validation set

```
In [ ]: print("Before UpSampling, counts of label '1': {}".format(sum(y_val == 1)))
print("Before UpSampling, counts of label '0': {} \n".format(sum(y_val == 0)))

sm = SMOTE(
    sampling_strategy=1, k_neighbors=5, random_state=1
) # Synthetic Minority Over Sampling Technique
X_val_over, y_val_over = sm.fit_resample(X_val, y_val.ravel())

print("After UpSampling, counts of label '1': {}".format(sum(y_val_over == 1)))
print("After UpSampling, counts of label '0': {} \n".format(sum(y_val_over == 0)))

print("After UpSampling, the shape of train_X: {}".format(X_val_over.shape))
print("After UpSampling, the shape of train_y: {} \n".format(y_val_over.shape))
```

Before UpSampling, counts of label '1': 326
Before UpSampling, counts of label '0': 1700

After UpSampling, counts of label '1': 1700
After UpSampling, counts of label '0': 1700

After UpSampling, the shape of train_X: (3400, 29)
After UpSampling, the shape of train_y: (3400,)

Sample tuning method for Decision tree with original data

```
In [ ]: # defining model
Model = DecisionTreeClassifier(random_state=1)

# Parameter grid to pass in RandomSearchCV
param_grid = {'max_depth': np.arange(2,6),
              'min_samples_leaf': [1, 4, 7],
              'max_leaf_nodes' : [10,15],
              'min_impurity_decrease': [0.0001,0.001] }

#Calling RandomizedSearchCV
randomized_cv = RandomizedSearchCV(estimator=Model, param_distributions=param_grid, n_iter=10, n_jobs = -1, sco

#Fitting parameters in RandomizedSearchCV
randomized_cv.fit(X_train,y_train)

print("Best parameters are {} with CV score={}" .format(randomized_cv.best_params_,randomized_cv.best_score_))
```

GradientBoost with oversampled

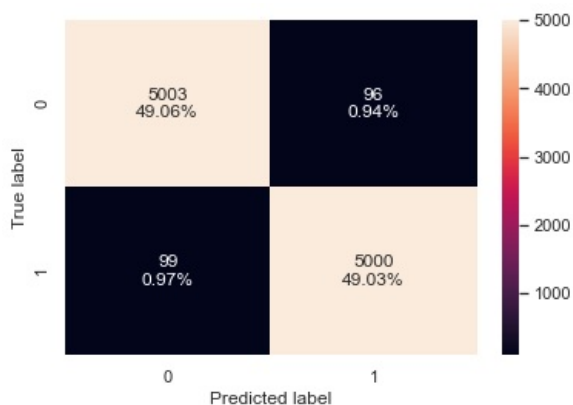
```
In [ ]: # fit the model
model = GradientBoostingClassifier(random_state=1)
gbost = model.fit(X_train_over, y_train_over)
```

```
In [ ]: # Calculating different metrics on train set
gradient_over = model_performance_classification_sklearn(
    gbost, X_train_over, y_train_over
)
print("Training performance:")
gradient_over
```

Training performance:

```
Out[ ]:   Accuracy  Recall  Precision   F1
0      0.981   0.981    0.981  0.981
```

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(gbost, X_train_over, y_train_over)
```



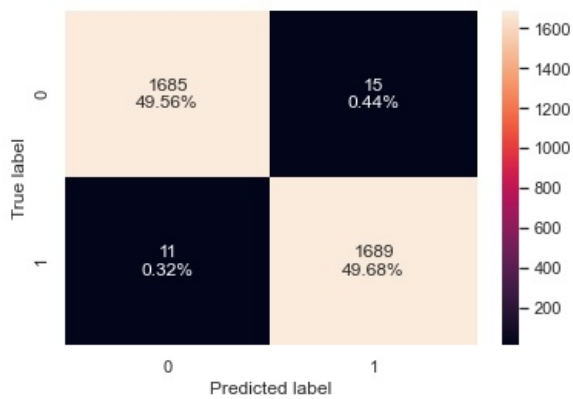
```
In [ ]: # fit the model
model = GradientBoostingClassifier(random_state=1)
gbosv = model.fit(X_val_over, y_val_over)
```

```
In [ ]: # Calculating different metrics on train set
gradient_over_val = model_performance_classification_sklearn(
    gbosv, X_val_over, y_val_over
)
print("Validation performance:")
gradient_over_val
```

Validation performance:

```
Out[ ]:   Accuracy  Recall  Precision   F1
0      0.992   0.994    0.991  0.992
```

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(gbosv, X_val_over, y_val_over)
```



AdaBoost with oversampled

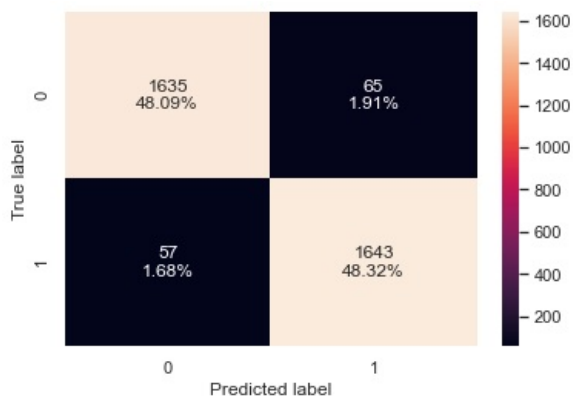
```
In [ ]: # fit the model
model = AdaBoostClassifier(random_state=1)
adost = model.fit(X_train_over, y_train_over)
```

```
In [ ]: # Calculating different metrics on train set
Adaboost_over = model_performance_classification_sklearn(
    adost, X_train_over, y_train_over
)
print("Training performance:")
Adaboost_over
```

Training performance:

```
Out[ ]: Accuracy Recall Precision F1
0      0.968  0.969    0.967 0.968
```

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(adost, X_val_over, y_val_over)
```



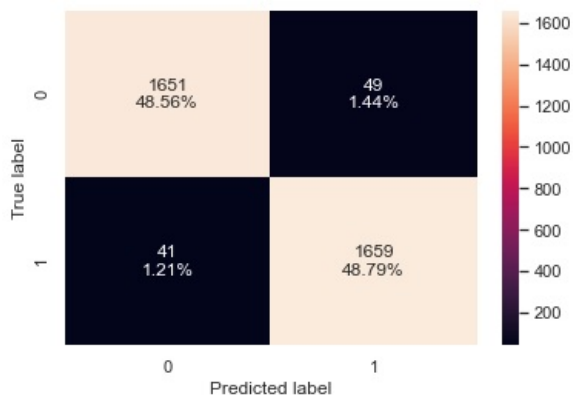
```
In [ ]: # fit the model
model = AdaBoostClassifier(random_state=1)
adosv = model.fit(X_val_over, y_val_over)
```

```
In [ ]: # Calculating different metrics on train set
Adaboost_over_val = model_performance_classification_sklearn(
    adosv, X_val_over, y_val_over
)
print("Validation performance:")
Adaboost_over_val
```

Validation performance:

```
Out[ ]: Accuracy Recall Precision F1
0      0.974  0.976    0.971 0.974
```

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(adost, X_val_over, y_val_over)
```



XGBoost with oversampled

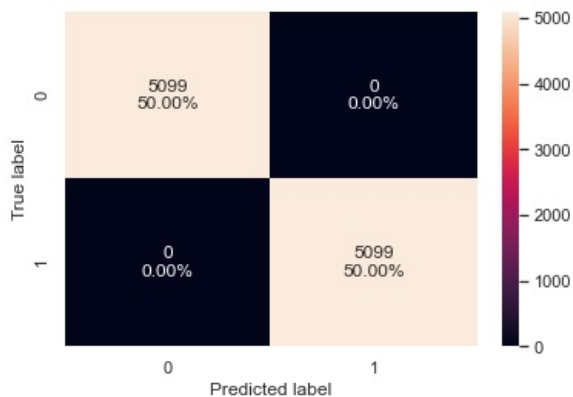
```
In [ ]: # defining model
model = XGBClassifier(random_state=1, eval_metric="logloss")
xgbost = model.fit(X_train_over, y_train_over)
```

```
In [ ]: # Calculating different metrics on train set
xgboost_over = model_performance_classification_sklearn(
    xgbost, X_train_over, y_train_over
)
print("Training performance:")
xgboost_over
```

Training performance:

```
Out[ ]: Accuracy Recall Precision F1
0 1.000 1.000 1.000 1.000
```

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(xgbost, X_train_over, y_train_over)
```



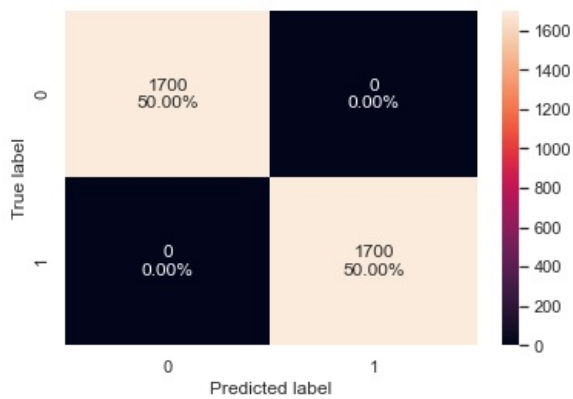
```
In [ ]: # fit the model
model = XGBClassifier(random_state=1, eval_metric="logloss")
xgbosv = model.fit(X_val_over, y_val_over)
```

```
In [ ]: # Calculating different metrics on train set
xgboost_over_val = model_performance_classification_sklearn(
    xgbosv, X_val_over, y_val_over
)
print("Validation performance:")
xgboost_over_val
```

Validation performance:

```
Out[ ]: Accuracy Recall Precision F1
0 1.000 1.000 1.000 1.000
```

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(xgbosv, X_val_over, y_val_over)
```



UnderSampling with Tuned Models

Building the best modeling using understampled data on the train and validation sets.

Training set

```
In [ ]: undersample = RandomUnderSampler(sampling_strategy=0.5)
X_under, y_under = undersample.fit_resample(X_train, y_train)
```

```
In [ ]: display(X_train.shape)
display(X_under.shape)

(6075, 29)
(2928, 29)
```

Validation set

```
In [ ]: undersample = RandomUnderSampler(sampling_strategy=0.5)
X_under_val, y_under_val = undersample.fit_resample(X_val, y_val)
```

```
In [ ]: display(X_val.shape)
display(X_under_val.shape)

(2026, 29)
(978, 29)
```

GradientBoost with undersampled

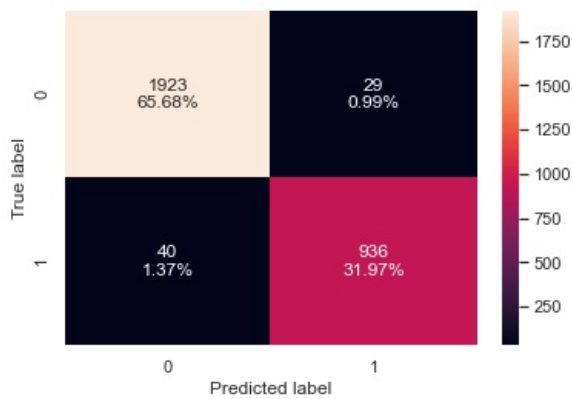
```
In [ ]: # fit the model
model = GradientBoostingClassifier(random_state=1)
gbust = model.fit(X_under, y_under)
```

```
In [ ]: # Calculating different metrics on train set
gradient_under = model_performance_classification_sklearn(gbust, X_under, y_under)
print("Training performance:")
gradient_under
```

Training performance:

```
Out[ ]:   Accuracy  Recall  Precision    F1
0      0.976   0.959    0.970  0.964
```

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(gbust, X_under, y_under)
```



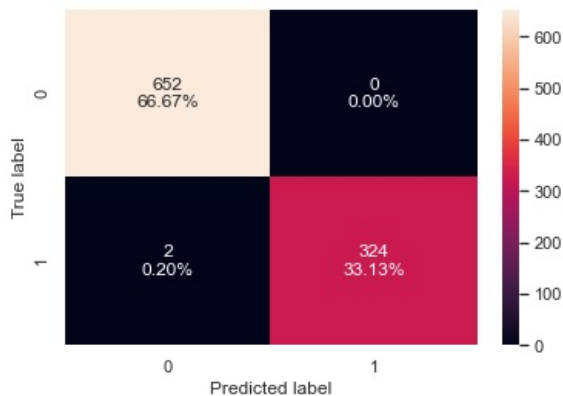
```
In [ ]: # fit the model
model = GradientBoostingClassifier(random_state=1)
gbusv = model.fit(X_under_val, y_under_val)
```

```
In [ ]: # Calculating different metrics on train set
gradient_under_val = model_performance_classification_sklearn(
    gbusv, X_under_val, y_under_val
)
print("Validation performance:")
gradient_under_val
```

Validation performance:

```
Out[ ]:   Accuracy  Recall  Precision   F1
0      0.998   0.994    1.000  0.997
```

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(gbusv, X_under_val, y_under_val)
```



AdaBoost with undersampled

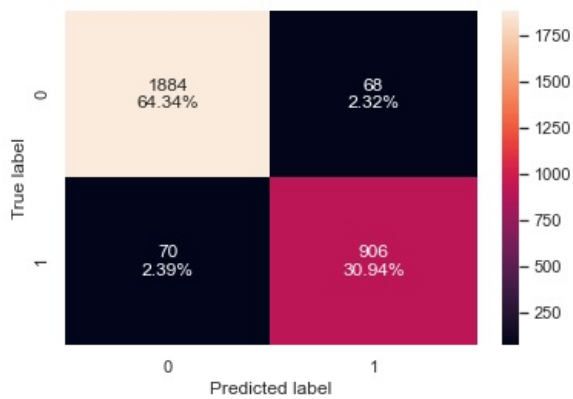
```
In [ ]: # fit the model
model = AdaBoostClassifier(random_state=1)
adust = model.fit(X_under, y_under)
```

```
In [ ]: # Calculating different metrics on train set
Adaboost_under = model_performance_classification_sklearn(adust, X_under, y_under)
print("Training performance:")
Adaboost_under
```

Training performance:

```
Out[ ]:   Accuracy  Recall  Precision   F1
0      0.953   0.928    0.930  0.929
```

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(adust, X_under, y_under)
```



```
In [ ]: # fit the model
model = AdaBoostClassifier(random_state=1)
adusv = model.fit(X_under_val, y_under_val)
```

```
In [ ]: # Calculating different metrics on train set
Adaboost_under_val = model_performance_classification_sklearn(
    adusv, X_under_val, y_under_val
)
print("Validation performance:")
Adaboost_under_val
```

Validation performance:

```
Out[ ]: Accuracy Recall Precision F1
0      0.974   0.957   0.966 0.961
```

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(adusv, X_under_val, y_under_val)
```



XGBoost with undersampled

```
In [ ]: # defining model
model = XGBClassifier(random_state=1, eval_metric="logloss")
xgbust = model.fit(X_under, y_under)
```

```
In [ ]: display(xgbust)
```

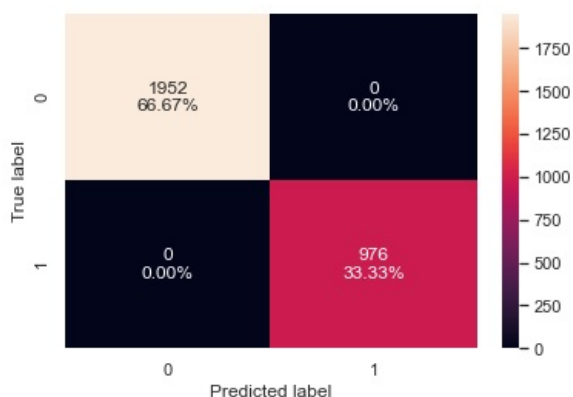
```
XGBClassifier(base_score=0.5, booster='gbtree', colsample_bylevel=1,
              colsample_bynode=1, colsample_bytree=1, eval_metric='logloss',
              gamma=0, gpu_id=-1, importance_type='gain',
              interaction_constraints='', learning_rate=0.300000012,
              max_delta_step=0, max_depth=6, min_child_weight=1, missing=nan,
              monotone_constraints=(), n_estimators=100, n_jobs=4,
              num_parallel_tree=1, random_state=1, reg_alpha=0, reg_lambda=1,
              scale_pos_weight=1, subsample=1, tree_method='exact',
              validate_parameters=1, verbosity=None)
```

```
In [ ]: # Calculating different metrics on train set
xgboost_under = model_performance_classification_sklearn(xgbust, X_under, y_under)
print("Training performance:")
xgboost_under
```

Training performance:

```
Out[ ]: Accuracy Recall Precision F1
0 1.000 1.000 1.000 1.000
```

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(xgbust, X_under, y_under)
```



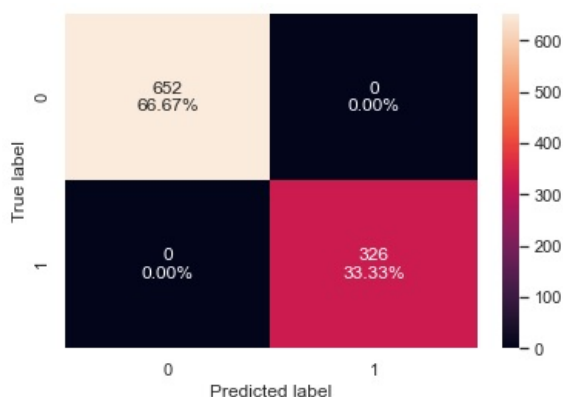
```
In [ ]: # defining model
model = XGBClassifier(random_state=1, eval_metric="logloss")
xgbusv = model.fit(X_under_val, y_under_val)
```

```
In [ ]: # Calculating different metrics on train set
xgboost_under_val = model_performance_classification_sklearn(
    xgbusv, X_under_val, y_under_val
)
print("Validation performance:")
xgboost_under_val
```

Validation performance:

```
Out[ ]: Accuracy Recall Precision F1
0 1.000 1.000 1.000 1.000
```

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(xgbusv, X_under_val, y_under_val)
```



Model Selection

```
In [ ]: # concatenate the dataframes
models_train_comp_df = pd.concat(
    [
        gradient_grid_train.T,
        gradient_grid_val.T,
        gradient_random_train.T,
        gradient_random_val.T,
        gradient_over.T,
        gradient_over_val.T,
        gradient_under.T,
        gradient_under_val.T,
        Adaboost_grid_train.T,
        Adaboost_grid_val.T,
        Adaboost_random_train.T,
        Adaboost_random_val.T,
        Adaboost_over.T,
        Adaboost_over_val.T,
    ]
)
```

```

        Adaboost_under.T,
        Adaboost_under_val.T,
        xgboost_grid_train.T,
        xgboost_grid_val.T,
        xgboost_random_train.T,
        xgboost_random_val.T,
        xgboost_over.T,
        xgboost_over_val.T,
        xgboost_under.T,
        xgboost_under_val.T,
    ],
    axis=1,
)
# give the columns headers
models_train_comp_df.columns = [
    "GradientBoost Tuned with Grid search Training Set",
    "GradientBoost Tuned with Grid search Validation Set",
    "GradientBoost Tuned with Random search Training Set",
    "GradientBoost Tuned with Random search Validation Set",
    "GradientBoost Tuned with Oversampled Training Set",
    "GradientBoost Tuned with Oversampled Validation Set",
    "GradientBoost Tuned with Undersampled Training Set",
    "GradientBoost Tuned with Undersampled Validation Set",
    "AdaBoost Tuned with Grid search Training Set",
    "AdaBoost Tuned with Grid search Validation Set",
    "AdaBoost Tuned with Random search Training Set",
    "AdaBoost Tuned with Random search Validation Set",
    "AdaBoost Tuned with Oversampled Training Set",
    "AdaBoost Tuned with Oversampled Validation Set",
    "AdaBoost Tuned with Undersampled Training Set",
    "AdaBoost Tuned with Undersampled Validation Set",
    "Xgboost Tuned with Grid search Training Set",
    "Xgboost Tuned with Grid search Validation Set",
    "Xgboost Tuned with Random Search Training Set",
    "Xgboost Tuned with Random Search Validation Set",
    "Xgboost Tuned with Oversampled Training Set",
    "Xgboost Tuned with Oversampled Validation Set",
    "Xgboost Tuned with Undersampled Training Set",
    "Xgboost Tuned with Undersampled Validation Set",
]

# print the dataframe
print("Training & Validation performance comparison:")
models_train_comp_df

```

Training & Validation performance comparison:

Out[]:

	GradientBoost Tuned with Grid search Training Set	GradientBoost Tuned with Grid search Validation Set	GradientBoost Tuned with Random search Training Set	GradientBoost Tuned with Random search Validation Set	GradientBoost Tuned with Oversampled Training Set	GradientBoost Tuned with Oversampled Validation Set	GradientBoost Tuned with Undersampled Training Set	GradientBoost Tuned with Undersampled Validation Set	AdaBoost Tuned with Grid search Training Set
Accuracy	0.986	0.973	0.982	0.972	0.981	0.992	0.976	0.998	0.9
Recall	0.941	0.896	0.929	0.883	0.981	0.994	0.959	0.994	0.9
Precision	0.973	0.936	0.957	0.941	0.981	0.991	0.970	1.000	0.9
F1	0.957	0.915	0.943	0.911	0.981	0.992	0.964	0.997	0.9

In []: # create a dataframe from the confusion matrix false negative scores

```

data = {
    "Gradient Boost Tuned with Gridsearch Training Set": 58,
    "Gradient Boost Tuned with Gridsearch Validation Set": 34,
    "Gradient Boost Tuned with Randomsearch Training Set": 69,
    "Gradient Boost Tuned with Randomsearch Validation Set": 38,
    "Gradient Boost Tuned with Oversampled Training Set": 99,
    "Gradient Boost Tuned with Oversampled Validation Set": 11,
    "Gradient Boost Tuned with Undersampled Training Set": 40,
    "Gradient Boost Tuned with Undersampled Validation Set": 2,
    "AdaBoost Tuned with Gridsearch Training Set": 49,
    "AdaBoost Tuned with Gridsearch Validation Set": 36,
    "AdaBoost Tuned with Randomsearch Training Set": 12,
    "AdaBoost Tuned with Randomsearch Validation Set": 40,
    "AdaBoost Tuned with Oversampled Training Set": 57,
    "AdaBoost Tuned with Oversampled Validation Set": 41,
    "AdaBoost Tuned with Undersampled Training Set": 70,
    "AdaBoost Tuned with Undersampled Validation Set": 16,
    "Xgboost Tuned with Gridsearch Training Set": 21,
    "Xgboost Tuned with Gridsearch Validation Set": 3,
    "Xgboost Tuned with RandomSearch Training Set": 13,
    "Xgboost Tuned with RandomSearch Validation Set": 12,
    "Xgboost Tuned with Oversampled Training Set": 0,
    "Xgboost Tuned with Oversampled Validation Set": 0,
    "Xgboost Tuned with Undersampled Training Set": 0,
    "Xgboost Tuned with Undersampled Validation Set": 0,
}

```

```

}
false_negatives_df_train = pd.DataFrame.from_dict(
    data, orient="index", columns=["False Negatives"]
)
display(false_negatives_df_train.sort_values("False Negatives"))

```

	False Negatives
Xgboost Tuned with Undersampled Validation Set	0
Xgboost Tuned with Oversampled Validation Set	0
Xgboost Tuned with Oversampled Training Set	0
Xgboost Tuned with Undersampled Training Set	0
Gradient Boost Tuned with Undersampled Validation Set	2
Xgboost Tuned with Gridsearch Validation Set	3
Gradient Boost Tuned with Oversampled Validation Set	11
Xgboost Tuned with RandomSearch Validation Set	12
AdaBoost Tuned with Randomsearch Training Set	12
Xgboost Tuned with RandomSearch Training Set	13
AdaBoost Tuned with Undersampled Validation Set	16
Xgboost Tuned with Gridsearch Training Set	21
Gradient Boost Tuned with Gridsearch Validation Set	34
AdaBoost Tuned with Gridsearch Validation Set	36
Gradient Boost Tuned with Randomsearch Validation Set	38
AdaBoost Tuned with Randomsearch Validation Set	40
Gradient Boost Tuned with Undersampled Training Set	40
AdaBoost Tuned with Oversampled Validation Set	41
AdaBoost Tuned with Gridsearch Training Set	49
AdaBoost Tuned with Oversampled Training Set	57
Gradient Boost Tuned with Gridsearch Training Set	58
Gradient Boost Tuned with Randomsearch Training Set	69
AdaBoost Tuned with Undersampled Training Set	70
Gradient Boost Tuned with Oversampled Training Set	99

Model Comparison

- The Recall Score is above 85%.
- Most of the models are not overfitting.
- False Negatives are low for all the models.

Performance on the test set

Choose the model that has Recall & Precision > 0.95 and find test accuracy > 0.70 on test set.

```

In [ ]: # Calculating different metrics on the test set
gbsov_test = model_performance_classification_sklearn(gbosv, X_test, y_test)
print("GradientBoost Tuned with Oversampled Validation Set: Test performance:")
gbsov_test

```

GradientBoost Tuned with Oversampled Validation Set: Test performance:

```

Out[ ]:   Accuracy  Recall  Precision    F1
0      0.954   0.874     0.843  0.858

```

```

In [ ]: # Calculating different metrics on the test set
adaboost_test = model_performance_classification_sklearn(adb_tuned2, X_test, y_test)
print("AdaBoost Tuned with Random search Training Set: Test performance:")
adaboost_test

```

AdaBoost Tuned with Random search Training Set: Test performance:

```

Out[ ]:   Accuracy  Recall  Precision    F1
0      0.976   0.929     0.921  0.925

```

```

In [ ]: # Calculating different metrics on the test set
gbusv_test = model_performance_classification_sklearn(gbusv, X_test, y_test)
print("GradientBoost Tuned with Undersampled Validation Set: Test performance:")

```

```
gbusv_test
```

GradientBoost Tuned with Undersampled Validation Set: Test performance:

```
Out[ ]:
```

	Accuracy	Recall	Precision	F1
0	0.956	0.911	0.829	0.868

None of these models performed above 95% on Recall and Precision and above 70% in accuracy. Let's go back to the comparison frame and try different models.

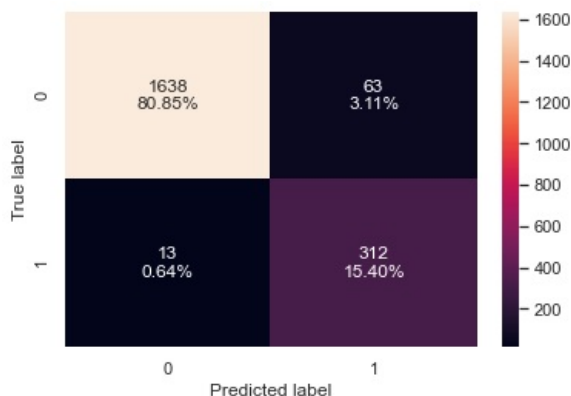
```
In [ ]: # Calculating different metrics on the test set
xgbust_test = model_performance_classification_sklearn(xgbust, X_test, y_test)
print("XGBoost with UnderSampled Training Data: Test performance:")
xgbust_test
```

XGBoost with UnderSampled Training Data: Test performance:

```
Out[ ]:
```

	Accuracy	Recall	Precision	F1
0	0.962	0.960	0.832	0.891

```
In [ ]: # creating confusion matrix
confusion_matrix_sklearn(xgbust, X_test, y_test)
```



- The XGBoost with UnderSampled Training Data gives 3 success measures.
- The performance on test data is lower than validation.

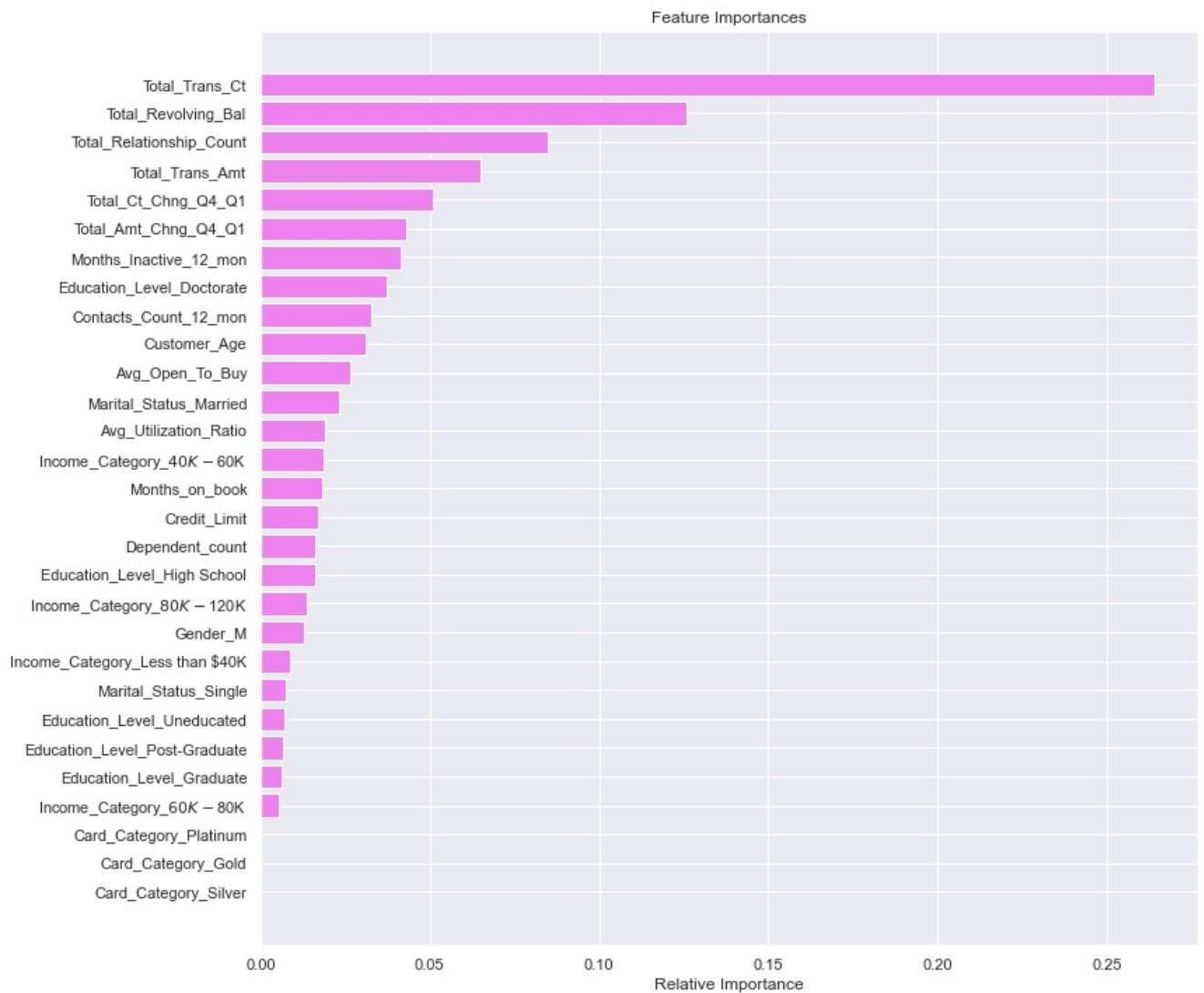
This model is giving us the accuracy, recall and precision we need, along with the low false negative error rate, so that we can use it in production.

Features of Importance

Let's extract the importance ranking from this model so as to point to where to focus our analysis efforts when examining credit card churners.

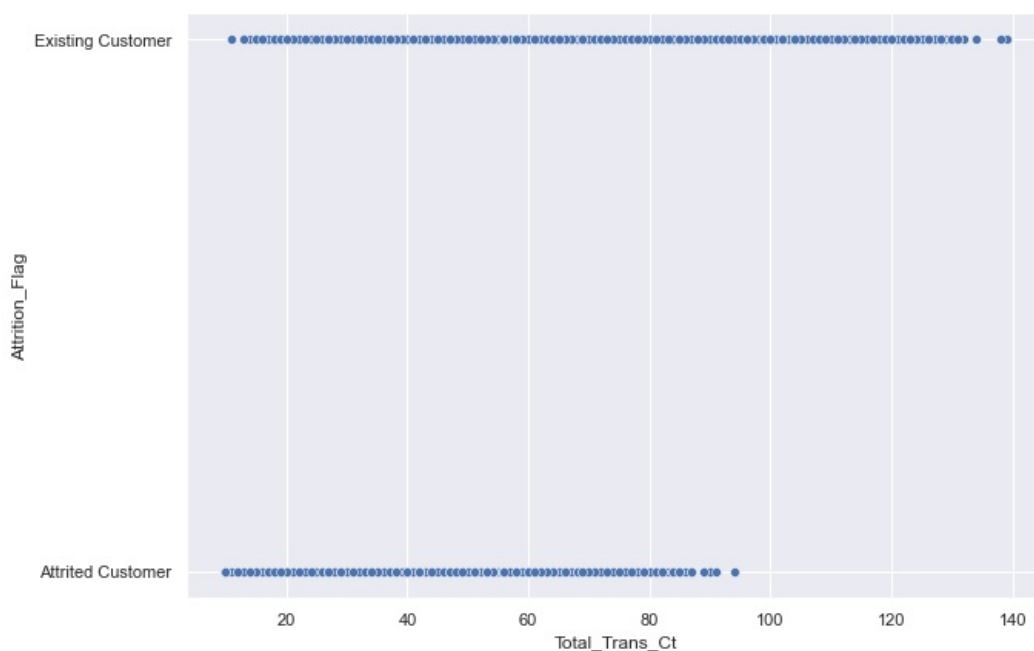
```
In [ ]: # List of features from dmodel
feature_names = X.columns
importances = xgbust.feature_importances_
indices = np.argsort(importances)

# plot the features
plt.figure(figsize=(12, 12))
plt.title("Feature Importances")
plt.barh(range(len(indices)), importances[indices], color="violet", align="center")
plt.yticks(range(len(indices)), [feature_names[i] for i in indices])
plt.xlabel("Relative Importance")
plt.show()
```



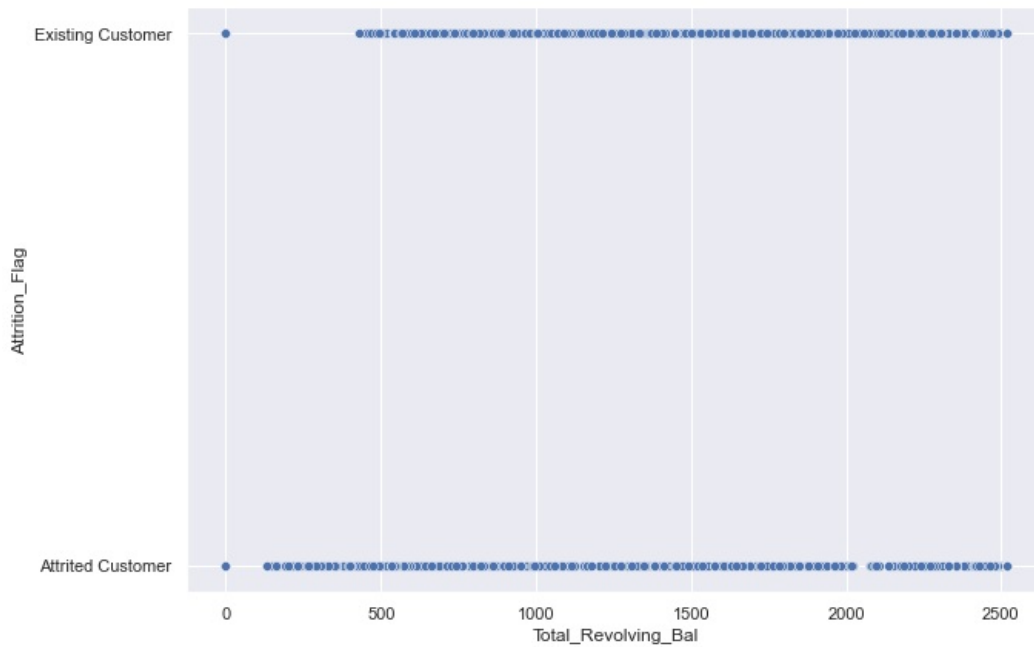
The total count of transactions is #1, followed by the total revolving monthly balance, then the total products count of the customer with the bank as the important features in predicting whether a customer will attrit or not.

```
In [ ]: # create a scatterplot
sns.scatterplot(x=df.Total_Trans_Ct, y=df.Attrition_Flag)
plt.show()
```



```
In [ ]: sns.scatterplot(x=df.Total_Revolving_Bal, y=df.Attrition_Flag)

Out[ ]: <AxesSubplot:xlabel='Total_Revolving_Bal', ylabel='Attrition_Flag'>
```

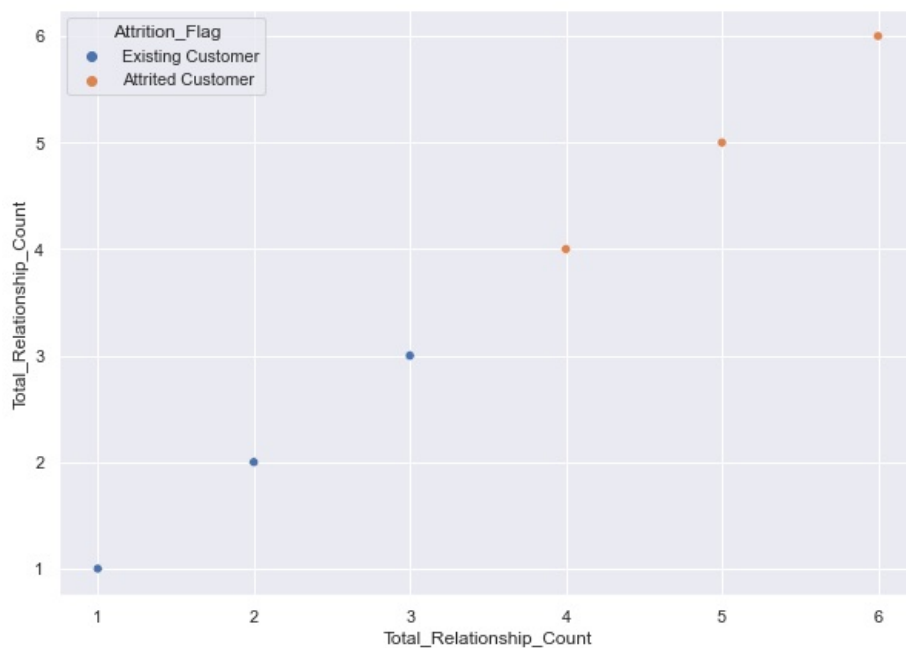


```
In [ ]: # crosstabs
pd.crosstab(index=df["Attrition_Flag"], columns=df["Total_Relationship_Count"])
```

```
Out[ ]: Total_Relationship_Count    1     2     3     4     5     6
Attrition_Flag
Attrited Customer      233   346   400   225   227   196
Existing Customer     677   897  1905  1687  1664  1670
```

```
In [ ]: sns.scatterplot(
    x=df.Total_Relationship_Count,
    y=df.Total_Relationship_Count,
    hue=df["Attrition_Flag"],
)
```

```
Out[ ]: <AxesSubplot:xlabel='Total_Relationship_Count', ylabel='Total_Relationship_Count'>
```



```
In [ ]: test_data = df.copy()
type(data)
test_data.head()
```

Out[]:	CLIENTNUM	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category
0	768805383	Existing Customer	45	M	3	High School	Married	60K – 80K	Blue
1	818770008	Existing Customer	49	F	5	Graduate	Single	Less than \$40K	Blue
2	713982108	Existing Customer	51	M	3	Graduate	Married	80K – 120K	Blue
3	769911858	Existing Customer	40	F	4	High School	NaN	Less than \$40K	Blue
4	709106358	Existing Customer	40	M	3	Uneducated	Married	60K – 80K	Blue

```
In [ ]: # Separating target variable and other variables
X = test_data.drop(columns=["Attrition_Flag"])
Y = test_data["Attrition_Flag"]
```

```
In [ ]: # Splitting the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, Y, test_size=0.30, random_state=1, stratify=Y
)
print(X_train.shape, X_test.shape)
display(X_train.head())
display(y_train)
```

(7088, 20) (3039, 20)

	CLIENTNUM	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category	Months_on
678	710280558	51	M	1	Graduate	Single	60K – 80K	Blue	
7524	713920158	41	F	3	Uneducated	NaN	abc	Blue	
8725	824177733	53	F	2	High School	Single	Less than \$40K	Blue	
10029	715268508	42	M	3	Graduate	Married	40K – 60K	Gold	
1383	710872233	27	M	0	NaN	Single	40K – 60K	Blue	

678 Existing Customer
7524 Attrited Customer
8725 Existing Customer
10029 Existing Customer
1383 Existing Customer

...
509 Attrited Customer
8365 Existing Customer
7169 Existing Customer
6113 Attrited Customer
10034 Attrited Customer

Name: Attrition_Flag, Length: 7088, dtype: object

```
In [ ]: # Transforming targetting variable to int.
target_variable = "Attrition_Flag"

y_train.replace(
    to_replace={"Attrited Customer": 1, "Existing Customer": 0}
)

# to drop unnecessary columns
drop_cols = "CLIENTNUM"
drop_transformer = X_train.drop(columns=drop_cols, inplace=True)
```

```
In [ ]: # creating a list of numerical variables
numerical_features = [
    "Customer_Age",
    "Dependent_count",
    "Months_on_book",
    "Total_Relationship_Count",
    "Months_Inactive_12_mon",
    "Contacts_Count_12_mon",
    "Credit_Limit",
    "Total_Revolving_Bal",
    "Avg_Open_To_Buy",
    "Total_Amt_Chng_Q4_Q1",
    "Total_Trans_Amt",
    "Total_Trans_Ct",
    "Total_Ct_Chng_Q4_Q1",
    "Avg_Utilization_Ratio",
]
```

```

numeric_transformer = Pipeline(steps=[("imputer", SimpleImputer(strategy="median"))])
categorical_features = [
    "Gender",
    "Education_Level",
    "Marital_Status",
    "Income_Category",
    "Card_Category",
]

categorical_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")), # handle missing
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]
)

preprocessor = ColumnTransformer(
    transformers=[ # List of (name, transformer, columns)
        ("num_step", numeric_transformer, numerical_features),
        ("cat_step", categorical_transformer, categorical_features),
    ],
    remainder="passthrough",
    n_jobs=-1,
    verbose=True,
)
#

```

```

In [ ]: # New pipeline with important parameters
production_model = Pipeline(
    steps=[
        ("pre", preprocessor),
        (
            "XGB",
            XGBClassifier(
                base_score=0.5,
                booster="gbtree",
                colsample_bylevel=1,
                colsample_bynode=1,
                colsample_bytree=1,
                eval_metric="logloss",
                gamma=0,
                gpu_id=-1,
                importance_type="gain",
                interaction_constraints="",
                learning_rate=0.300000012,
                max_delta_step=0,
                max_depth=6,
                min_child_weight=1,
                missing=np.nan,
                monotone_constraints="()",
                n_estimators=100,
                n_jobs=4,
                num_parallel_tree=1,
                random_state=1,
                reg_alpha=0,
                reg_lambda=1,
                scale_pos_weight=1,
                subsample=1,
                tree_method="exact",
                validate_parameters=1,
                verbosity=None,
            ),
        ),
    ]
)

```

```

In [ ]: # viewing pipeline
display(production_model[0])
display(type(production_model))

```

```

ColumnTransformer(n_jobs=-1, remainder='passthrough',
                  transformers=[('num_step',
                                Pipeline(steps=[('imputer',
                                                  SimpleImputer(strategy='median'))]),
                                ['Customer_Age', 'Dependent_count',
                                'Months_on_book', 'Total_Relationship_Count',
                                'Months_Inactive_12_mon',
                                'Contacts_Count_12_mon', 'Credit_Limit',
                                'Total_Revolving_Bal', 'Avg_Open_To_Buy',
                                'Total_Amt_Chng_Q4_Q1', 'Total_Trans_Amt',
                                'Total_Trans_Ct', 'Total_Ct_Chng_Q4_Q1',
                                'Avg_Utilization_Ratio'])],
                  ('cat_step',
                   Pipeline(steps=[('imputer',
                                     SimpleImputer(strategy='most_frequent')),
                                     ('onehot',
                                      OneHotEncoder(handle_unknown='ignore'))]),
                   ['Gender', 'Education_Level', 'Marital_Status',
                   'Income_Category', 'Card_Category'])],
                  verbose=True)
sklearn.pipeline.Pipeline

```

Fitting the production pipeline

```

In [ ]: # Fit the model on training data
production_model.fit(X_train, y_train.values.ravel())

```

```

Out[ ]: Pipeline(steps=[('pre',
                          ColumnTransformer(n_jobs=-1, remainder='passthrough',
                                              transformers=[('num_step',
                                                            Pipeline(steps=[('imputer',
                                                                              SimpleImputer(strategy='median'))]),
                                                            ['Customer_Age',
                                                            'Dependent_count',
                                                            'Months_on_book',
                                                            'Total_Relationship_Count',
                                                            'Months_Inactive_12_mon',
                                                            'Contacts_Count_12_mon',
                                                            'Credit_Limit',
                                                            'Total_Revolving_Bal',
                                                            'Avg_Open_To_Buy',
                                                            'Tot...
gamma=0, gpu_id=-1, importance_type='gain',
interaction_constraints='',
learning_rate=0.300000012, max_delta_step=0,
max_depth=6, min_child_weight=1, missing=nan,
monotone_constraints='()', n_estimators=100,
n_jobs=4, num_parallel_tree=1, random_state=1,
reg_alpha=0, reg_lambda=1, scale_pos_weight=1,
subsample=1, tree_method='exact',
validate_parameters=1, verbosity=None)))

```

Business Insights and Conclusions

- The customers who attrit based on their historical customer profile should be targetted by sending offers via online or by mail.
- Additionally, the bank should not offer better deals to customers whose total product holding is in between 2 to 3.
- Moreover, the bank should not offer better deals to customers whose monthly balance is below 500.
- For low-priority customers, the bank can offer different credit cards with different limits.